

Curso: Minimal APIs no ASP.NET Core 10

Oct 5, 2026 · @Daniel Freire

Como usar este curso

Você vai sair daqui sabendo construir, em ASP.NET Core 10, uma API HTTP de produção com Minimal APIs: contrato explícito em OpenAPI, erros padronizados, validação, segurança, testes de integração reais e observabilidade, organizada de um jeito que continua legível com 200 endpoints.

O curso segue o mesmo método do curso de processamento de arquivos: **top-down**. Primeiro o que a API precisa garantir, depois o caminho que uma requisição percorre dentro do framework, depois cada peça desse caminho e, só então, o código. Cada módulo responde a uma pergunta de design e termina com um exercício que vira parte do projeto final.

Pré-requisitos

C# intermediário, `async/await`, injeção de dependência e noções de HTTP (verbos, status codes, headers). Não é preciso ter usado ASP.NET Core antes, mas quem já usou Controllers vai encontrar comparações ao longo do caminho.

Exemplo condutor

O exemplo é a **API de cobrança**, a porta de entrada do pipeline do primeiro curso. Ela recebe uploads de arquivos de retorno, informa o status de cada arquivo, consulta títulos e ocorrências e transmite o progresso de um processamento em tempo real. É um bom exemplo porque mistura os casos difíceis: upload grande, consultas paginadas sobre tabelas enormes, idempotência, autorização por cliente e streaming.

Stack usada

- .NET 10 + C# 14, ASP.NET Core 10 com Minimal APIs
- EF Core 10 + Npgsql sobre PostgreSQL
- `Microsoft.AspNetCore.OpenApi` para o documento OpenAPI e Scalar como interface de exploração
- xUnit, `WebApplicationFactory` e Testcontainers para testes
- OpenTelemetry para métricas, traces e logs

Mapa dos módulos

Módulo	Pergunta que responde	O que você entrega
1. O problema	O que uma API de produção precisa garantir?	Uma lista de requisitos da sua API
2. Arquitetura	Que caminho uma requisição percorre?	O mapa do pipeline com seus middlewares
3. Fundamentos	Como declaro rotas e recebo dados?	Os primeiros endpoints com binding correto
4. Respostas e erros	Como devolvo sucesso e falha de forma previsível?	Respostas tipadas, ProblemDetails e validação
5. Organização	Como o código cresce sem virar um <code>Program.cs</code> gigante?	Estrutura por feature com filtros
6. Dados	Como ler, gravar e receber arquivos sem travar nada?	Paginação keyset, streaming e upload
7. Contrato	Como o cliente sabe o que a API faz?	Documento OpenAPI fiel e versionamento
8. Segurança	Quem pode chamar o quê, e quantas vezes?	JWT, políticas, rate limiting e CORS
9. Performance	Como responder rápido sob carga?	Cache, compressão, timeouts e SSE
10. Testes	Como provo que tudo funciona?	Testes de integração com banco real
11. Operação	Como sei que está saudável em produção?	Health checks, telemetria e deploy
Projeto final	Tudo junto funciona?	A API de cobrança completa

Sugestão de ritmo: um módulo por semana. Os Módulos 3, 4 e 5 são a base de todo o resto e merecem mais tempo; os Módulos 6 e 10 são onde o curso mais se conecta com o de processamento de arquivos.

Módulo 1 — O problema: o que uma API de produção precisa garantir

Uma API é duas coisas ao mesmo tempo: um **contrato** (o que cada rota aceita e devolve, inclusive nos erros) e um conjunto de **promessas operacionais** (quem pode chamar, quantas vezes, com que latência e o que acontece se a chamada for repetida). Minimal APIs são só a forma de escrever; este módulo define o que precisa ser escrito.

1.1 As garantias de uma API de produção

Um endpoint que "funciona no Postman" cumpre talvez uma destas linhas. Uma API de produção cumpre todas:

Garantia	Pergunta que ela responde	Onde o curso trata
Contrato explícito	O cliente sabe exatamente o que mandar e o que vai receber?	Módulos 4 e 7
Erros previsíveis	Toda falha tem o mesmo formato e um status code correto?	Módulo 4
Entrada validada	Dados inválidos são barrados antes de chegar na regra de negócio?	Módulo 4
Autorização	Cada cliente só vê e altera o que é dele?	Módulo 8
Idempotência	Repetir uma chamada por timeout causa efeito duplicado?	Módulos 1 e 6
Limites	Um cliente pode derrubar a API para os outros?	Módulos 8 e 9
Desempenho	A latência continua aceitável no pico?	Módulos 6 e 9
Evolução	Dá para mudar sem quebrar quem já integra?	Módulo 7
Operabilidade	Dá para saber o que aconteceu com uma requisição específica?	Módulo 11

1.2 HTTP com significado

Boa parte do contrato já está definida pelo próprio HTTP. Respeitar a semântica dos verbos faz proxies, caches, retries automáticos e desenvolvedores se comportarem do jeito certo sem documentação extra.

Verbo	Seguro (não altera estado)	Idempotente	Uso típico
GET	Sim	Sim	Ler um recurso ou uma lista
POST	Não	Não	Criar recurso, disparar ação
PUT	Não	Sim	Substituir o recurso inteiro
PATCH	Não	Depende	Alterar parte do recurso
DELETE	Não	Sim	Remover

Idempotente significa que repetir a mesma requisição produz o mesmo estado final. O ponto crítico é o **POST**: se o cliente envia um arquivo, a conexão cai antes da resposta e ele tenta de novo, o servidor pode ter criado o recurso duas vezes. A solução padrão é o header **Idempotency-Key**: o cliente gera um identificador único por operação, o servidor grava a chave junto com o resultado e, se a mesma chave chegar de novo, devolve o resultado gravado em vez de executar outra vez. Você vai implementar isso no Módulo 6.

Os status codes que uma API de negócio realmente usa são poucos, e cada um tem um significado preciso:

- **200 OK**: sucesso com corpo. **201 Created**: recurso criado, com header **Location** apontando para ele. **202 Accepted**: aceito para processamento posterior (o caso do upload de arquivo, que é processado de forma assíncrona). **204 No Content**: sucesso sem corpo.
- **400 Bad Request**: requisição malformada ou inválida. **401 Unauthorized**: não autenticado (o nome engana). **403 Forbidden**: autenticado, mas sem permissão. **404 Not Found**: não existe, ou existe mas o cliente não pode saber que existe. **409 Conflict**: conflito de estado (por exemplo, arquivo duplicado). **422 Unprocessable Content**: bem formada, mas viola regra de negócio. **429 Too Many Requests**: passou do limite.
- **500**: erro não tratado no servidor. Um 500 é sempre um bug ou uma dependência fora do ar, nunca uma resposta de negócio.

1.3 Minimal APIs x Controllers

Minimal APIs surgiram no .NET 6 como uma forma de declarar endpoints como funções, sem classes de controller. Desde então ganharam quase tudo que faltava: route groups, filtros, respostas tipadas, geração de OpenAPI e, no .NET 10, validação embutida. Hoje são o modelo recomendado para APIs novas.

Aspecto	Minimal APIs	Controllers (MVC)
Unidade de código	Um handler (função ou método estático) por endpoint	Uma classe com várias actions
Dependências	Injetadas por parâmetro, só as que o endpoint usa	Injetadas no construtor, compartilhadas por todas as actions
Filtros	Endpoint filters, com acesso tipado aos argumentos	Action, resource, result e exception filters
Tipagem de resposta	<code>TypedResults</code> e <code>Results<T1, T2></code> , verificados pelo compilador	<code>ActionResult<T></code> e atributos <code>[ProducesResponseType]</code>
Native AOT	Suportado, via gerador de código de request delegates	Não suportado
Sobrecarga por requisição	Menor	Maior (pipeline do MVC)
Quando ainda escolher	Praticamente sempre para APIs novas	Base existente grande em MVC, ou dependência forte de recursos específicos do MVC (OData, model binders customizados complexos)

O argumento mais comum contra Minimal APIs, "vira um `Program.cs` gigante", é um problema de organização, não do modelo. O Módulo 5 inteiro existe para resolver isso.

1.4 Os requisitos da API de cobrança

O projeto final terá estes endpoints. Leia a tabela pensando em cada garantia da seção 1.1:

Método e rota	O que faz	Detalhe que torna difícil
<code>POST /v1/arquivos</code>	Recebe um arquivo de retorno	Upload grande em streaming, idempotência, 202
<code>GET /v1/arquivos/{id}</code>	Status do processamento	Autorização: só o dono vê
<code>GET /v1/arquivos/{id}/progresso</code>	Progresso em tempo real	Server-Sent Events

Método e rota	O que faz	Detalhe que torna difícil
GET /v1/arquivos	Lista arquivos do cliente	Paginação keyset, filtros
GET /v1/titulos/{nossoNumero}	Consulta um título	Cache, 404 correto
GET /v1/titulos/{nossoNumero}/ocorrencias	Histórico do título	Lista potencialmente longa
GET /v1/ocorrencias/export	Exporta ocorrências do dia	Resposta em streaming de milhões de linhas

Requisitos não funcionais de exemplo: p95 abaixo de 200 ms para consultas, uploads de até 500 MB, 100 requisições por minuto por cliente, autenticação por JWT emitido por um provedor de identidade externo.

Exercício do Módulo 1

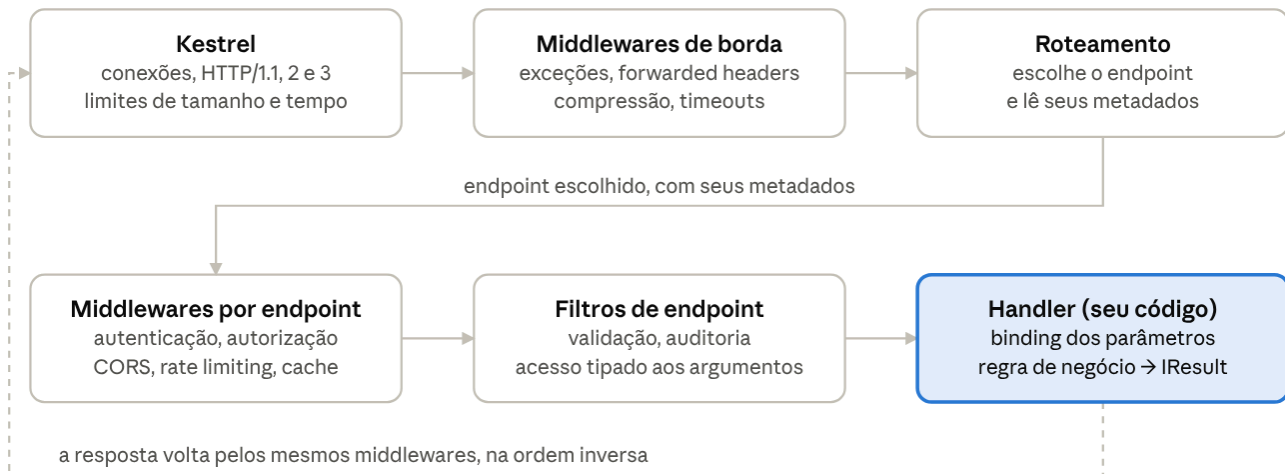
Pegue uma API que você mantém (ou a API de cobrança) e preencha a tabela 1.1 para ela: para cada garantia, escreva como ela é cumprida hoje ou "não é". Depois revise os verbos e status codes de cada endpoint contra a seção 1.2. Normalmente aparecem `POST` usados para consulta, `200` com `{ "erro": ... }` no corpo e `500` para validação. Guarde a lista; ela vira o backlog do curso.

Módulo 2 — Arquitetura: o caminho de uma requisição

Toda requisição passa pelo mesmo caminho: o Kestrel aceita a conexão, uma cadeia de middlewares trata o que vale para todos, o roteamento escolhe o endpoint, middlewares que dependem do endpoint (autenticação, autorização, rate limiting) decidem se ela segue, filtros de endpoint inspecionam os argumentos e, por último, o seu handler roda. Entender esse caminho explica quase todos os bugs de configuração de uma API.

Toda requisição atravessa o mesmo pipeline antes de chegar ao seu código

Caminho de uma requisição no ASP.NET Core 10 - a ordem de cima para baixo é a ordem de execução



pipeline de requisição · 6 etapas, resposta no caminho inverso

A seta tracejada é tão importante quanto as cheias: o middleware que roda primeiro na ida é o último na volta. É por isso que o tratador de exceções fica no topo: ele envolve tudo o que vem depois e enxerga qualquer exceção que suba.

2.1 Kestrel: a porta de entrada

O Kestrel é o servidor HTTP embutido do ASP.NET Core. Em produção ele normalmente fica atrás de um proxy reverso (um ingress do Kubernetes, um load balancer da nuvem), que termina o TLS e repassa a requisição. Duas consequências práticas:

- **Limites:** o Kestrel impõe por padrão um corpo máximo de cerca de 30 MB por requisição (`MaxRequestBodySize`). Para o upload de arquivos de 500 MB, você vai aumentar esse limite **só no endpoint de upload** (Módulo 6), nunca globalmente.
- **Headers encaminhados:** atrás do proxy, o IP de origem e o esquema (`https`) que o Kestrel enxerga são os do proxy. O middleware `ForwardedHeaders` lê `X-Forwarded-For` e `X-Forwarded-Proto` e corrige isso. Configure quais proxies são confiáveis; aceitar esses headers de qualquer origem permite que um cliente forje o próprio IP e escape do rate limiting.

O Kestrel também proíbe I/O síncrono por padrão (`AllowSynchronousIO = false`). Se algum código ler o corpo com `Read` em vez de `ReadAsync`, a requisição falha. Isso é proposital: I/O síncrono bloqueia threads do pool e derruba a capacidade do servidor sob carga.

2.2 Middlewares: a cadeia que envolve tudo

Um middleware é uma função que recebe o `HttpContext` e a referência para o próximo elo (`next`). Ele pode fazer algo antes, chamar `next` , fazer algo depois, ou não chamar `next` e responder ele mesmo (curto-circuito).

```
app.Use(async (contexto, next) =>
{
    long inicio = Stopwatch.GetTimestamp();           // antes: na ida

    await next(contexto);                             // todo o resto do
pipeline

    var duracao = Stopwatch.GetElapsedTime(inicio); // depois: na volta
    contexto.RequestServices.GetRequiredService<ILogger<Program>>()
        .LogInformation("{Metodo} {Rota} {Status} em {Ms} ms",
            contexto.Request.Method, contexto.Request.Path,
            contexto.Response.StatusCode, duracao.TotalMilliseconds);
});
```

Cuidado com o "depois": quando `next` retorna, a resposta pode já ter começado a ser enviada. Tentar mudar status code ou headers nesse ponto lança exceção. Para alterar headers da resposta, use `contexto.Response.OnStarting(...)` , que roda imediatamente antes do envio.

A ordem de registro é a ordem de execução. Esta é a ordem recomendada para uma API e o motivo de cada posição:


```

var app = builder.Build();

app.UseForwardedHeaders();    // 1. corrige IP e esquema antes que alguém os
leia
app.UseExceptionHandler();    // 2. envolve tudo que vem depois; converte
exceções em ProblemDetails
app.UseStatusCodePages();    // 3. dá corpo ProblemDetails a 404/405 que
saíam vazios
app.UseResponseCompression(); // 4. comprime o que voltar
app.UseRequestTimeouts();    // 5. cancela requisições que passam do prazo
app.UseCors();                // 6. precisa saber o endpoint; antes da
autenticação (preflight não tem token)
app.UseAuthentication();      // 7. descobre quem é o usuário
app.UseAuthorization();        // 8. decide se ele pode acessar este endpoint
app.UseRateLimiter();          // 9. depois da autenticação, para poder
limitar por cliente
app.UseOutputCache();          // 10. depois da autorização, para nunca
servir cache a quem não pode ver

app.MapGrupoArquivos();
app.MapGrupoTitulos();

app.Run();

```

Você não viu `UseRouting()` nessa lista. O `WebApplication` o insere automaticamente no início do pipeline quando há endpoints mapeados, e por isso CORS, autorização e rate limiting já sabem qual endpoint foi escolhido quando rodam. Se você chamar `UseRouting()` explicitamente, ele roda na posição em que foi chamado, e tudo que depende do endpoint precisa vir depois dele.

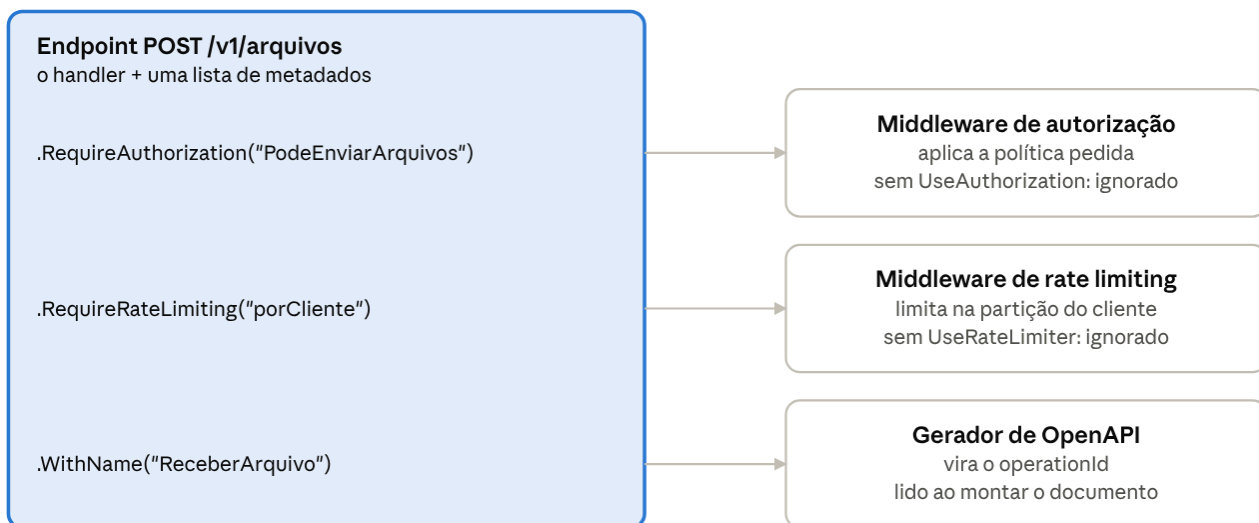
2.3 Endpoints e metadados

Cada `MapGet`, `MapPost` etc. cria um **endpoint**: um `RequestDelegate` (a função que atende a requisição) mais uma coleção de **metadados**. Métodos como `.RequireAuthorization("Cliente")`, `.RequireRateLimiting("porCliente")`, `.CacheOutput()` e `.WithName("ObterArquivo")` não executam nada sozinhos: eles só adicionam metadados ao endpoint.

Quem lê esses metadados são os middlewares. O de autorização pergunta ao endpoint escolhido "você tem uma política?" e aplica. O gerador de OpenAPI percorre todos os endpoints e lê nomes, tipos de resposta e descrições. Esse desenho tem uma consequência importante: se o middleware não estiver no pipeline, o metadado é ignorado em silêncio. Um `.RequireRateLimiting(...)` sem `app.UseRateLimiter()` não limita nada e não gera erro.

Os métodos só anotam o endpoint; quem age é o middleware que lê a anotação

Metadados de um endpoint e quem os lê



metadados de endpoint · 3 anotações, 3 leitores

Os três metadados ficam no endpoint de qualquer jeito; o que muda o comportamento é haver alguém no pipeline para lê-los.

2.4 Middleware ou filtro de endpoint?

Critério	Middleware	Filtro de endpoint
Alcance	Todas as requisições (ou as que passam por ele)	Um endpoint ou um grupo
Enxerga	<code>HttpContext</code> cru	<code>HttpContext</code> + argumentos já convertidos do handler
Roda	Antes do roteamento ou depois dele	Depois do binding, imediatamente antes do handler
Bom para	Exceções, compressão, correlação, autenticação	Validação específica, auditoria, regras de acesso a um recurso

Regra prática: se a lógica precisa dos argumentos tipados do endpoint ("o `arquivoId` pertence a este cliente?"), é filtro. Se vale para tudo e não se importa com o endpoint, é middleware.

2.5 Como o seu lambda vira um RequestDelegate

Você escreve um handler com parâmetros tipados:

```
app.MapGet("/v1/arquivos/{id:guid}", async (Guid id, IRepositorioDeArquivos
repo, CancellationToken ct) => ...);
```

Mas o pipeline só entende `RequestDelegate`, uma função `Task (HttpContext)`. Alguém precisa gerar o código que lê `id` da rota, converte para `Guid`, resolve o repositório do DI, pega o `CancellationToken` da requisição, chama o lambda e escreve o resultado. Existem dois caminhos para isso:

- **Em tempo de execução** (padrão): na subida da aplicação, o `RequestDelegateFactory` inspeciona o lambda por reflexão e compila esse código. Funciona sempre, custa um pouco de tempo de subida.
- **Em tempo de compilação**: o **Request Delegate Generator** (RDG), um gerador de código C#, produz esse código durante o build. É obrigatório para Native AOT e pode ser ligado sem AOT com `<EnableRequestDelegateGenerator>true</EnableRequestDelegateGenerator>`. A subida fica mais rápida e erros de binding aparecem como avisos de compilação.

Entender isso desmistifica o binding (Módulo 3): não há mágica, só regras que decidem de onde vem cada parâmetro.

2.6 Injeção de dependência e escopos

Cada requisição cria um **escopo** de DI. Serviços registrados como `Scoped` (o `DbContext`, por exemplo) vivem exatamente uma requisição e são descartados no fim dela.

`Singleton` vive enquanto a aplicação viver e é compartilhado por todas as requisições simultâneas, então precisa ser thread-safe. `Transient` é criado a cada resolução.

O erro clássico é a **dependência cativa**: um singleton que recebe um serviço scoped no construtor. O scoped passa a viver para sempre e é usado por várias requisições ao mesmo tempo (um `DbContext` compartilhado entre threads corrompe estado). Em ambiente `Development`, o host valida escopos e lança exceção na subida; mantenha `ValidateScopes` e `ValidateOnBuild` ligados nos testes também.

Outra regra: o `HttpContext` não é thread-safe e só é válido durante a requisição. Nunca o guarde num campo nem o use dentro de um `Task.Run` que pode terminar depois da resposta.

Exercício do Módulo 2

Crie um projeto `dotnet new web`, registre três middlewares que escrevem no log "entrando A/B/C" e "saindo C/B/A" e um endpoint simples. Confirme a ordem no console. Depois coloque `UseAuthorization()` antes de um `UseRouting()` explícito, marque o

endpoint com `.RequireAuthorization()` e observe que a autorização deixa de funcionar. Volte à ordem correta e explique por escrito, em duas frases, o que mudou.

Módulo 3 — Fundamentos: rotas, grupos e binding

Um endpoint de Minimal API é três coisas: um **template de rota** com restrições, um **handler** cujos parâmetros dizem de onde vêm os dados, e **convenções** (nome, tags, autorização) herdadas do grupo em que ele está. Dominar as regras de binding é o que separa um endpoint que "às vezes recebe null" de um que nunca surpreende.

3.1 O host: `WebApplicationBuilder` e `WebApplication`

O `Program.cs` tem duas fases bem separadas. Na primeira, o `builder` registra serviços e configuração; nada está rodando ainda. Na segunda, o `app` monta o pipeline e os endpoints; a partir daqui não dá mais para registrar serviços.

```
var builder = WebApplicationBuilder.CreateBuilder(args);

// Fase 1: serviços
builder.Services.AddProblemDetails();
builder.Services.AddValidation();
builder.Services.AddOpenApi();

var app = builder.Build();

// Fase 2: pipeline e endpoints
app.UseExceptionHandler();
app.UseStatusCodePages();

app.MapOpenApi();
app.MapGrupoArquivos();

app.Run();
```

A configuração é lida em camadas, e a última ganha: `appsettings.json`, depois `appsettings.{Ambiente}.json`, depois user secrets (só em `Development`), variáveis de ambiente e argumentos de linha de comando. Em Kubernetes, variáveis de ambiente com `__` (dois sublinhados) mapeiam seções: `ConnectionStrings__Cobranca` vira `ConnectionStrings:Cobranca`.

Para Native AOT existe `WebApplication.CreateSlimBuilder(args)`, que registra menos coisas por padrão (sem HTTPS, sem alguns provedores de log). Você vai vê-lo no Módulo 9.

3.2 Rotas e restrições

Um template de rota tem partes fixas e parâmetros entre chaves. **Restrições** filtram quais valores casam com a rota:

```
app.MapGet("/v1/arquivos/{id:guid}", ...);           // só GUIDs
app.MapGet("/v1/titulos/{nossoNumero:long:min(1)}", ...); // inteiro
longo positivo
app.MapGet("/v1/ocorrencias/{data:datetime}", ...);
app.MapGet("/v1/arquivos/{id:guid}/linhas/{*resto}", ...); // catch-all
```

Uma restrição que falha faz a rota **não casar**, o que resulta em 404, e não em 400. Isso é correto para identificadores (um `id` que não é GUID realmente não identifica nada), mas para validação de negócio prefira validação explícita (Módulo 4), que devolve um 400 com explicação. Restrições são para desambiguar rotas, não para validar entrada.

Dê nome aos endpoints que outros vão referenciar: `.WithName("ObterArquivo")`. O nome serve para gerar URLs (o header `Location` de um 201 ou 202) e vira o `operationId` no OpenAPI.

3.3 Route groups: convenções em um só lugar

`MapGroup` cria um prefixo de rota e aplica convenções a todos os endpoints dentro dele. É a ferramenta principal de organização:

```
RouteGroupBuilder v1 = app.MapGroup("/v1")
    .RequireAuthorization()           // todo /v1 exige usuário autenticado
    .AddEndpointFilter<CorrelacaoFilter>();

RouteGroupBuilder arquivos = v1.MapGroup("/arquivos")
    .WithTags("Arquivos")           // agrupa no OpenAPI
    .RequireRateLimiting("porCliente");

arquivos.MapGet("/{id:guid}", ObterArquivo).WithName("ObterArquivo");
arquivos.MapPost("/",
    ReceberArquivo).RequireAuthorization("PodeEnviarArquivos");
```

Grupos aninhados acumulam convenções: o `POST` acima exige autenticação (do `/v1`), a política `PodeEnviarArquivos` (dele mesmo), o rate limiting e o filtro de correlação. Isso elimina a categoria inteira de bugs "esqueci o `[Authorize]` neste endpoint": a autorização é o padrão do grupo, e a exceção precisa ser explícita (`.AllowAnonymous()`).

No Módulo 5, cada feature monta o próprio grupo com o caminho completo (`/v1/arquivos`) e as convenções dele, em vez de pendurá-lo num grupo `/v1` externo. O aninhamento acima mostra o mecanismo; as duas formas funcionam, desde que o projeto

escolha uma e a use em todo lugar. O resto do curso usa a do Módulo 5.

3.4 Model binding: de onde vem cada parâmetro

O framework decide a origem de cada parâmetro do handler. Você pode ser explícito com atributos, ou deixar a inferência agir. As regras de inferência, na ordem em que são aplicadas:

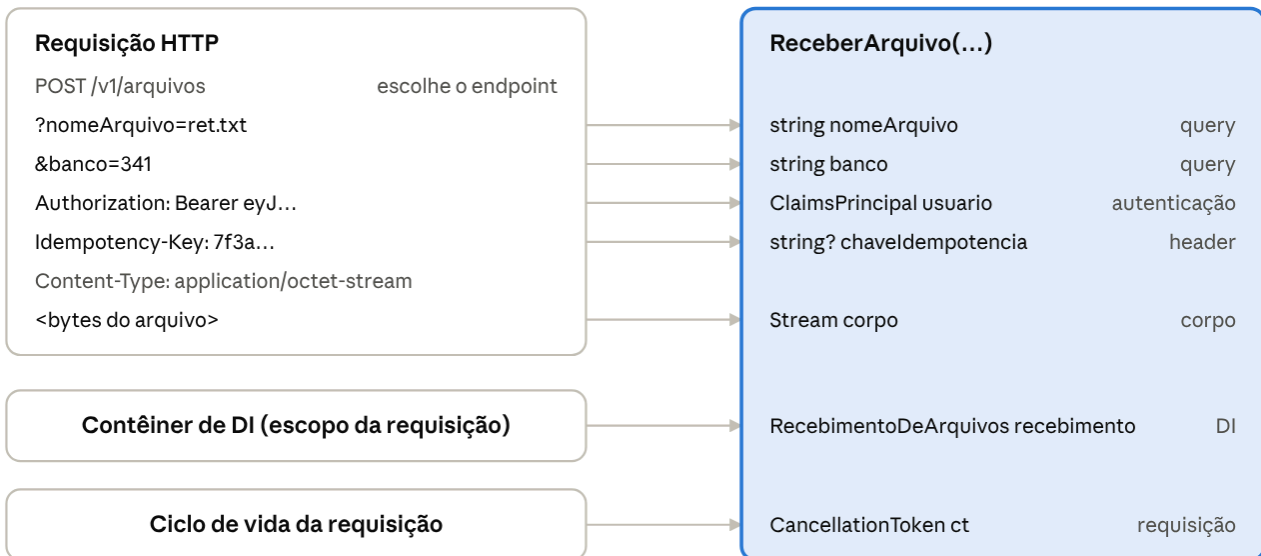
1. **Tipos especiais** são injetados diretamente: `HttpContext`, `HttpRequest`, `HttpResponse`, `CancellationToken` (ligado ao cancelamento da requisição), `ClaimsPrincipal` (o usuário), `IFormFile` / `IFormFileCollection`, `Stream` e `PipeReader` (o corpo cru).
2. **Serviços registrados no DI** são resolvidos do escopo da requisição.
3. **Tipos simples** (primitivos, `Guid`, `DateOnly`, enums e qualquer tipo com `TryParse` estático) vêm da **rota** se houver um parâmetro de rota com o mesmo nome; senão, da **query string**.
4. **Tipos complexos** vêm do **corpo** como JSON (somente um parâmetro pode vir do corpo).

Atributo explícito	Origem	Exemplo
<code>[FromRoute]</code>	Segmento da rota	<code>/ {id}</code>
<code>[FromQuery]</code>	Query string	<code>?status=Concluido</code>
<code>[FromHeader(Name = "Idempotency-Key")]</code>	Header	<code>Idempotency-Key: 7f3a...</code>
<code>[FromBody]</code>	Corpo JSON	<code>{ "descricao": ... }</code>
<code>[FromForm]</code>	Formulário (<code>multipart/form-data</code> ou <code>urlencoded</code>)	Campos de upload
<code>[FromServices]</code> / <code>[FromKeyedServices("chave")]</code>	Contêiner de DI	Um serviço por chave
<code>[AsParameters]</code>	Agrupa vários parâmetros num tipo	Filtros de listagem

Recomendação: seja explícito para headers e formulários (não há inferência para eles) e para qualquer caso ambíguo. A inferência é ótima para rota, query e serviços.

Cada parâmetro do handler vem de uma parte da requisição ou do DI

Binding do POST /v1/arquivos (Módulo 6.5) · o tipo e o nome decidem a origem



binding do upload · 7 parâmetros, 5 origens

As duas linhas sem seta não viram parâmetro: o caminho escolhe o endpoint, e o `Content-Type` só aparece no contrato, via `.Accepts(...)`.

Nulabilidade importa: um parâmetro `string status` (não anulável) é obrigatório, e a ausência gera 400. `string? status` é opcional. Parâmetros com valor padrão (`int limite = 50`) também são opcionais. Com `<Nullable>enable</Nullable>`, o seu modelo de tipos vira o contrato.

3.5 Agrupando parâmetros com [AsParameters]

Endpoints de listagem acumulam filtros. Em vez de um handler com oito parâmetros, agrupe-os num `record`:

```
public sealed record FiltroDeArquivos(
    [FromQuery] StatusArquivo? Status,
    [FromQuery(Name = "de")] DateOnly? De,
    [FromQuery(Name = "ate")] DateOnly? Ate,
    [FromQuery] string? Cursor,
    [FromQuery] int Limite = 50);

arquivos.MapGet("/", async ([AsParameters] FiltroDeArquivos filtro,
    ClaimsPrincipal usuario,
    IConsultaDeArquivos consulta,
    CancellationToken ct) =>
    TypedResults.Ok(await consulta.ListarAsync(usuario.ClienteId(), filtro,
    ct))); // esboço: a versão final, com cursor, está no Módulo 6.3
```

O `record` vira um tipo de domínio da consulta: pode ser passado adiante, testado e validado (Módulo 4) sem carregar o `HttpContext` junto.

3.6 Binding customizado: TryParse e BindAsync

Qualquer tipo seu pode vir da rota ou da query se tiver um método estático `TryParse`. A forma mais limpa é implementar `IParsable<T>`. Isso permite usar **value objects** direto na assinatura do endpoint:

```
public readonly record struct NossoNumero(long Valor) :
    IParsable<NossoNumero>
{
    public static bool TryParse(string? s, IFormatProvider? provider, out
        NossoNumero resultado)
    {
        resultado = default;
        if (string.IsNullOrEmpty(s) || s.Length > 20) return false;
        foreach (char c in s)
            if (!char.IsAsciiDigit(c)) return false;
        if (!long.TryParse(s, NumberStyles.None,
            CultureInfo.InvariantCulture, out long valor) || valor == 0)
            return false;
        resultado = new NossoNumero(valor);
        return true;
    }

    public static NossoNumero Parse(string s, IFormatProvider? provider)
        => TryParse(s, provider, out var r) ? r : throw new
        FormatException("Nosso número inválido.");

    public override string ToString() =>
        Valor.ToString(CultureInfo.InvariantCulture);
}

app.MapGet("/v1/titulos/{nossoNumero}", (NossoNumero nossoNumero, ...) =>
    ...);
```

Se `TryParse` devolver `false`, o framework responde 400 automaticamente, antes de chamar o handler. O handler só vê valores válidos, e o tipo carrega esse significado pelo resto do código. Por isso esta rota não leva a restrição `:long` da seção 3.2: com a restrição, um valor inválido não casaria com a rota e viraria 404 em vez de 400.

Para binding que precisa do `HttpContext` inteiro (ler vários headers, por exemplo), implemente `static ValueTask<T?> BindAsync(HttpContext contexto)`. Use com moderação: é menos explícito para quem lê o endpoint.

3.7 JSON

Minimal APIs usam `System.Text.Json`. As opções globais ficam em `ConfigureHttpJsonOptions`:

```
builder.Services.ConfigureHttpJsonOptions(o =>
{
    o.SerializerOptions.Converters.Add(new JsonStringEnumConverter()); //
enums como texto
    o.SerializerOptions.DefaultIgnoreCondition =
JsonIgnoreCondition.WhenWritingNull;
    o.SerializerOptions.TypeInfoResolverChain.Insert(0,
CobrancaJsonContext.Default); // source generation
});

[JsonSerializable(typeof(ArquivoResposta))]
[JsonSerializable(typeof(PaginaDe<ArquivoResumo>))]
internal partial class CobrancaJsonContext : JsonSerializerContext;
```

Três decisões de contrato aqui. O padrão web já usa **camelCase** nas propriedades; não mude. **Enums como texto** (`"Concluido"` em vez de `3`) tornam o contrato legível e resistente a reordenações do enum. E o **source generation** (`JsonSerializerContext`) gera o código de serialização em tempo de compilação: é obrigatório para Native AOT e reduz custo de subida e de memória mesmo sem AOT.

No .NET 10, a leitura do corpo JSON nos endpoints passou a usar `PipeReader` internamente, o que reduz cópias e alocações sem mudança no seu código.

3.8 Cancellation Token em todo handler assíncrono

O `CancellationToken` injetado num handler é cancelado quando o cliente desiste (fecha a conexão) ou quando o timeout da requisição estoura (Módulo 9). Repasse-o para toda chamada assíncrona: EF Core, `HttpClient`, S3. Sem isso, uma consulta pesada continua rodando no banco depois que ninguém mais espera a resposta, e sob carga isso vira um efeito cascata.

Exercício do Módulo 3

Crie o grupo `/v1/titulos` com o endpoint `GET /{nossoNumero}` usando o `NossoNumero` acima e um repositório em memória. Teste quatro chamadas: um número válido existente (200), um válido inexistente (404), `abc` (400) e `0` (400). Depois crie `GET /v1/arquivos` com `[AsParameters]` `FiltroDeArquivos` e confira que `?limite=x` gera 400 e que a ausência de `limite` usa 50.

Módulo 4 — Respostas e erros previsíveis

O contrato de um endpoint é o conjunto de **todas** as respostas que ele pode dar, não só a de sucesso. Em Minimal APIs você escreve esse conjunto no tipo de retorno com `Results<...>`, devolve cada caso com `TypedResults`, padroniza toda falha como **ProblemDetails** e deixa a validação embutida do .NET 10 barrar entrada inválida antes do handler.

4.1 Results x TypedResults

Existem duas fábricas de resposta com nomes quase iguais e uma diferença importante:

- `Results.Ok(valor)` devolve `IResult`. O compilador e o gerador de OpenAPI não sabem o que tem dentro.
- `TypedResults.Ok(valor)` devolve `Ok<T>`, um tipo concreto. Ele implementa `IEndpointMetadataProvider`: só de aparecer no tipo de retorno, o endpoint anuncia ao OpenAPI "posso responder 200 com um `T`".

Quando um endpoint tem mais de um desfecho, declare a união no tipo de retorno:

```
public static async Task<Results<Ok<ArquivoResposta>, NotFound>>
ObterArquivo(
    Guid id, ClaimsPrincipal usuario, IConsultaDeArquivos consulta,
    CancellationToken ct)
{
    ArquivoResposta? arquivo = await consulta.ObterAsync(usuario.ClienteId(),
id, ct);

    return arquivo is null
        ? TypedResults.NotFound()
        : TypedResults.Ok(arquivo);
}
```

O que esse tipo de retorno compra:

1. **O compilador verifica o contrato.** Se alguém tentar devolver `TypedResults.Conflict()` daqui, não compila. Para adicionar um desfecho, é preciso mudá-lo na assinatura, onde todo revisor vê.
2. **O OpenAPI sai certo sem anotações.** As respostas 200 (com o schema de `ArquivoResposta`) e 404 aparecem no documento automaticamente.
3. **Testes de unidade ficam simples.** Você chama o método estático, faz pattern matching no resultado (`resultado.Result is Ok<ArquivoResposta> ok`) e verifica `ok.Value`, sem servidor HTTP.

Repare também no 404 para um arquivo que existe mas é de outro cliente. A consulta filtra pelo `ClienteId`, então para o cliente errado o arquivo simplesmente não existe. Responder 403 confirmaria que aquele ID existe, o que vaza informação.

4.2 Os desfechos de criação

Respostas de criação devem dizer onde o recurso está. O header `Location` é gerado a partir do nome do endpoint de consulta:

```
// Recurso criado e pronto: 201
return TypedResults.CreatedAtRoute(resposta, "ObterArquivo", new { id =
    resposta.Id });

// Recurso aceito para processamento assíncrono: 202
return TypedResults.AcceptedAtRoute(resposta, "ObterArquivo", new { id =
    resposta.Id });
```

O upload de arquivos de retorno é um caso clássico de **202 Accepted**: a API guarda o arquivo e responde na hora, o pipeline do primeiro curso processa depois, e o cliente acompanha pelo `Location` (ou pelo stream de progresso do Módulo 9).

4.3 ProblemDetails: um formato para todas as falhas

A RFC 9457 define um formato JSON padrão para erros HTTP. Adotá-lo significa que todo cliente trata erro do mesmo jeito, em qualquer endpoint:

```
{
  "type": "https://api.cobranca.exemplo/erros/arquivo-duplicado",
  "title": "Este arquivo já foi recebido.",
  "status": 409,
  "detail": "Um arquivo com o mesmo conteúdo foi recebido em 2026-10-05T08:12:44Z.",
  "instance": "POST /v1/arquivos",
  "traceId": "4bf92f3577b34da6a3ce929d0e0e4736",
  "arquivoExistenteId": "7c9e6679-7425-40de-944b-e07fc1f90ae7"
}
```

`type` é um identificador estável do tipo de erro (o cliente faz `switch` nele, nunca no texto). `title` é o resumo legível. `detail` explica esta ocorrência. Campos extras (`traceId`, `arquivoExistenteId`) são extensões permitidas pelo padrão.

Para que **toda** falha saia nesse formato, inclusive as que o framework gera (404 de rota inexistente, 405, exceções não tratadas), são três linhas:

```
builder.Services.AddProblemDetails(o => o.CustomizeProblemDetails = ctx =>
{
    ctx.ProblemDetails.Instance = $"{ctx.HttpContext.Request.Method}
{ctx.HttpContext.Request.Path}";
    ctx.ProblemDetails.Extensions["traceId"] =
        Activity.Current?.TraceId.ToString() ??
ctx.HttpContext.TraceIdentifier;
});

app.UseExceptionHandler(); // exceção não tratada → 500 em ProblemDetails
app.UseStatusCodePages(); // 404/405 sem corpo → ProblemDetails
```

O `traceId` em todo erro é o detalhe mais valioso para operação: o cliente manda o erro para o suporte, e o suporte encontra o trace completo da requisição (Módulo 11). Em produção, um 500 nunca deve expor a mensagem ou a pilha da exceção; o padrão fora de `Development` já é omitir.

4.4 Validação embutida no .NET 10

Até o .NET 9, quem usava Minimal APIs precisava escrever validação à mão ou usar bibliotecas externas. O .NET 10 trouxe validação nativa baseada em `DataAnnotations` para parâmetros de query, header, rota e corpo, inclusive em `records`. Ligue com uma linha:

```
builder.Services.AddValidation();
```

E declare as regras nos tipos:

```
public sealed record FiltroDeArquivos(
    [FromQuery] StatusArquivo? Status,
    [FromQuery(Name = "de")] DateOnly? De,
    [FromQuery(Name = "ate")] DateOnly? Ate,
    [FromQuery, StringLength(200)] string? Cursor,
    [FromQuery, Range(1, 200)] int Limite = 50) : IValidatableObject
{
    public IEnumerable<ValidationResult> Validate(ValidationContext contexto)
    {
        if (De is { } de && Ate is { } ate && de > ate)
            yield return new ValidationResult("'de' deve ser anterior ou
igual a 'ate'.", [nameof(De), nameof(Ate)]);

        if (De is { } d && Ate is { } a && a.DayNumber - d.DayNumber > 92)
            yield return new ValidationResult("O intervalo máximo é de 92
dias.", [nameof(De), nameof(Ate)]);
    }
}
```

```
public sealed record RegistrarCallbackRequest(
    [Required, Url, StringLength(500)] string Url,
    [Required, MinLength(1)] EventoDeCallback[] Eventos);
```

Quando a validação falha, o framework responde 400 com um `ValidationProblemDetails` (um `ProblemDetails` com um dicionário `errors` por campo), sem chamar o handler. A validação é feita por um gerador de código que descobre os tipos a partir das assinaturas dos endpoints, então não há reflexão em tempo de execução e ela é compatível com Native AOT. Para desligar a validação num endpoint específico, use `.DisableValidation()`.

Duas outras mudanças do .NET 10 que afetam a entrada: em formulários, um campo enviado vazio para um tipo anulável (como `DateOnly?`) agora vira `null` em vez de erro de conversão; e os 400 de validação passam pelo `IProblemDetailsService`, então recebem a mesma customização (`traceId`, `instance`) das outras falhas.

4.5 Validação não é regra de negócio

Separe dois tipos de verificação, porque cada um tem um status e um lugar:

	Validação de entrada	Regra de negócio
Pergunta	O pedido está bem formado?	O pedido faz sentido no estado atual?
Depende do banco	Não	Normalmente sim
Exemplo	<code>limite</code> entre 1 e 200; URL válida	Arquivo já recebido; título já liquidado
Status	400	409 (conflito de estado) ou 422 (violação de regra)
Onde fica	Atributos e <code>IValidatableObject</code>	Handler ou camada de domínio

Colocar regra de negócio em atributo de validação (um `[ArquivoNaoDuplicado]` que consulta o banco) é um erro comum: esconde I/O dentro da validação, não tem acesso à transação e não é à prova de concorrência. A verificação de duplicidade de verdade é o `UNIQUE` do banco.

4.6 Erros de negócio sem exceções

Desfechos esperados (duplicidade, estado inválido) não são excepcionais e não precisam de exceções. Modele-os como valores e converta para `ProblemDetails` num único lugar:

```
public abstract record ErroDeNegocio(string Codigo, string Titulo, int
Status);

public sealed record ArquivoDuplicado(Guid ExistenteId, DateTimeOffset
RecebidoEm)
    : ErroDeNegocio("arquivo-duplicado", "Este arquivo já foi recebido.",
StatusCodes.Status409Conflict);

public sealed record LayoutNaoSuportado(string Banco)
    : ErroDeNegocio("layout-nao-suportado", "Layout de arquivo não
suportado.", StatusCodes.Status422UnprocessableEntity);

public static class ErroDeNegocioExtensions
{
    public static ProblemHttpResult ParaProblema(this ErroDeNegocio erro) =>
TypedResults.Problem(
    type: $"https://api.cobranca.exemplo/erros/{erro.Codigo}",
    title: erro.Titulo,
    statusCode: erro.Status,
    extensions: erro switch
    {
        ArquivoDuplicado d => new Dictionary<string, object?> {
["arquivoExistenteId"] = d.ExistenteId },
        _ => null
    });
}
```

O handler fica explícito sobre seus desfechos:

```
public static async Task<Results<AcceptedAtRoute<ArquivoResposta>,
ProblemHttpResult>> ReceberArquivo(...)
{
    var resultado = await servico.ReceberAsync(...);
    return resultado.Erro is { } erro
        ? erro.ParaProblema()
        : TypedResults.AcceptedAtRoute(resultado.Valor!, "ObterArquivo", new
{ id = resultado.Valor!.Id });
}
```

4.7 Exceções inesperadas, mapeadas num lugar só

Algumas exceções de infraestrutura têm significado HTTP conhecido (um conflito de concorrência otimista do EF Core vira 409, por exemplo). Trate-as com um `ExceptionHandler`, que o `UseExceptionHandler` consulta antes de cair no 500 genérico:

```
public sealed class TratadorDeExcecoesConhecidas(IProblemDetailsService
problemas) : IExceptionHandler
{
    public async ValueTask<bool> TryHandleAsync(HttpContext contexto,
Exception excecao, CancellationToken ct)
    {
        (int Status, string Titulo)? mapeado = excecao switch
        {
            DbUpdateConcurrencyException => (StatusCodes.Status409Conflict,
"O recurso foi alterado por outra requisição."),
            BadRequestException bad => (bad.StatusCode, "Requisição
inválida."),
            _ => null
        };

        if (mapeado is null) return false; // segue para o 500 padrão

        contexto.Response.StatusCode = mapeado.Value.Status;
        return await problemas.TryWriteAsync(new ProblemDetailsContext
        {
            HttpContext = contexto,
            Exception = excecao,
            ProblemDetails = new ProblemDetails { Status =
mapeado.Value.Status, Title = mapeado.Value.Titulo }
        });
    }
}

builder.Services.AddExceptionHandler<TratadorDeExcecoesConhecidas>();
```

O `BadRequestException` aparece aqui porque é o que o framework lança quando o corpo excede o limite (413) ou o JSON é inválido (400); mapeando-o, esses casos também saem como `ProblemDetails` com o status certo em vez de 500.

Exercício do Módulo 4

Converta os endpoints do Módulo 3 para `Results<...>` com `TypedResults`. Ligue `AddProblemDetails` com `traceId`, `AddValidation` e o `ExceptionHandler`. Faça chamadas que produzam 400 (validação), 404 (rota inexistente e recurso inexistente), 409 (duplicidade simulada) e 500 (lance uma exceção qualquer) e confirme que as cinco respostas têm o mesmo formato e um `traceId`. Por fim, escreva um teste de unidade para `ObterArquivo` sem subir servidor.

Módulo 5 — Organização do código

Uma API com Minimal APIs se mantém legível com centenas de endpoints quando o código é organizado **por feature** (vertical slices): cada feature tem uma pasta com seus endpoints, contratos e serviços, expõe um método de extensão `MapGrupoX` para as rotas e outro `AddX` para o DI, e o `Program.cs` só liga as features. Regras transversais por endpoint ficam em filtros.

5.1 Estrutura de pastas por feature

```
Cobranca.Api/
├─ Program.cs                ~40 linhas: só composição
├─ Infraestrutura/
│   ├─ ProblemDetailsSetup.cs
│   ├─ AutenticacaoSetup.cs
│   └─ TelemetriaSetup.cs
├─ Comum/
│   ├─ ErroDeNegocio.cs
│   ├─ Paginacao.cs
│   └─ ClaimsPrincipalExtensions.cs
└─ Features/
    ├─ Arquivos/
    │   ├─ ArquivosEndpoints.cs    rotas e handlers
    │   ├─ ArquivosContratos.cs    requests e responses
    │   ├─ RecebimentoDeArquivos.cs regra do upload
    │   ├─ ConsultaDeArquivos.cs   leituras
    │   └─ ArquivosServicos.cs     AddArquivos()
    └─ Titulos/
        └─ ...
```

A vantagem sobre a organização por camada técnica (`Controllers/`, `Services/`, `Repositories/`) é que uma mudança de negócio toca uma pasta só. Quem vai alterar o upload abre `Features/Arquivos` e encontra tudo ali.

5.2 O padrão de uma feature

Cada feature é uma classe estática com um método de extensão público que monta o grupo e métodos privados que são os handlers:

```
public static class ArquivosEndpoints
{
    public static RouteGroupBuilder MapGrupoArquivos(this
IEndpointRouteBuilder rotas)
    {
        RouteGroupBuilder grupo = rotas.MapGroup("/v1/arquivos")
            .WithGroupName("v1") // documento OpenAPI desta
versão (Módulo 7)
            .WithTags("Arquivos")
            .RequireAuthorization()
            .RequireRateLimiting("porCliente");

        grupo.MapPost("/", ReceberArquivo)
            .WithName("ReceberArquivo")
            .WithSummary("Recebe um arquivo de retorno para processamento")
            .RequireAuthorization("PodeEnviarArquivos");

        grupo.MapGet("/", ListarArquivos).WithName("ListarArquivos");

        grupo.MapGet("/{id:guid}", ObterArquivo)
            .WithName("ObterArquivo")
            .ExigirDonoDoArquivo();

        return grupo;
    }

    // internal, e não private, para que os testes de unidade do Módulo 10
    possam chamá-lo
    internal static async Task<Results<Ok<ArquivoResposta>, NotFound>>
ObterArquivo(
        Guid id, ClaimsPrincipal usuario, IConsultaDeArquivos consulta,
        CancellationToken ct)
    {
        ArquivoResposta? arquivo = await
consulta.ObterAsync(usuario.ClienteId(), id, ct);
        return arquivo is null ? TypedResults.NotFound() :
TypedResults.Ok(arquivo);
    }
}
```

```
// ReceberArquivo (Módulo 6.5) e ListarArquivos (Módulo 6.3) seguem o
mesmo formato
}
```

E o `Program.cs` vira composição pura:

```
var builder = WebApplication.CreateBuilder(args);

builder.AddInfraestruturaDaApi(); // ProblemDetails, validação, OpenAPI,
auth, telemetria
builder.Services.AddArquivos();
builder.Services.AddTitulos();

var app = builder.Build();

app.UsePipelineDaApi(); // a ordem de middlewares do Módulo 2
app.MapGrupoArquivos();
app.MapGrupoTitulos();

app.Run();
```

Existe uma alternativa popular: uma interface `IEndpoint` com um método `MapEndpoint`, e uma varredura do assembly que registra todas as implementações automaticamente. Ela economiza linhas no `Program.cs`, mas usa reflexão (incompatível com Native AOT) e esconde o que está registrado. Com poucas dezenas de features, a lista explícita é mais clara.

5.3 Handlers finos, com nome

Prefira **métodos estáticos nomeados** a lambdas inline. Eles aparecem com nome em stack traces e no profiler, podem ser testados diretamente (Módulo 4) e mantêm o método `MapGrupoX` legível como um índice das rotas.

Um handler bem escrito faz só três coisas: recebe os parâmetros já convertidos e validados, chama **uma** operação (uma consulta ou um caso de uso), e traduz o desfecho para `TypedResults`. Se o handler tem `if` de regra de negócio, laço sobre dados ou acesso direto a duas tabelas, essa lógica pertence a um serviço.

5.4 Filtros de endpoint

Um filtro implementa `IEndpointFilter` e roda depois do binding e da validação, imediatamente antes do handler. Ele enxerga os argumentos já convertidos e pode curto-circuitar devolvendo um resultado:

```

public sealed class AuditoriaFilter(ILogger<AuditoriaFilter> log) :
    IEndpointFilter
{
    public async ValueTask<object?>
    InvokeAsync(EndpointFilterInvocationContext contexto,
                EndpointFilterDelegate next)
    {
        object? resultado = await next(contexto);    // executa o handler

        if (resultado is IStatusCodeHttpResult { StatusCode: >= 200 and < 300
    })
            log.LogInformation("Cliente {Cliente} executou {Endpoint}",
                                contexto.HttpContext.User.ClientId(),
                                contexto.HttpContext.GetEndpoint()?.DisplayName);

        return resultado;
    }
}

```

Quando o filtro precisa de um argumento específico, use uma **fábrica de filtro**. Ela roda uma vez por endpoint, na subida da aplicação, e pode inspecionar a assinatura do handler. Isso evita índices mágicos e faz erros de configuração aparecerem no boot, não na primeira requisição:

```

public static class DonoDoArquivoExtensions
{
    public static RouteHandlerBuilder ExigirDonoDoArquivo(this
    RouteHandlerBuilder builder) =>
        builder.AddEndpointFilterFactory((fabrica, next) =>
        {
            int indice = Array.FindIndex(fabrica.MethodInfo.GetParameters(),
                p => p.Name == "id" && p.ParameterType == typeof(Guid));

            if (indice < 0)
                throw new InvalidOperationException(
                    $"{fabrica.MethodInfo.Name} precisa de um parâmetro 'Guid
id' para usar ExigirDonoDoArquivo.");

            return async contexto =>
            {
                Guid id = contexto.GetArgument<Guid>(indice);
                var acesso =
contexto.HttpContext.RequestServices.GetRequiredService<IAcessoAArquivos>();
                bool pertence = await acesso.PertenceAsync(
                    contexto.HttpContext.User.ClientId(), id,

```

```

contexto.HttpContext.RequestAborted);

        return pertence ? await next(contexto) :
TypedResults.NotFound();
    };
});
}

```

Com esse filtro, a regra "só o dono vê o arquivo" fica declarada na rota (`.ExigirDonoDoArquivo()`) e pode ser aplicada a `/progresso`, `/linhas` e qualquer outra rota sob o arquivo. A consulta do handler continua filtrando pelo cliente: são duas defesas independentes, e o Módulo 8 explica por que vale manter as duas mesmo que pareçam redundantes.

Ordem: filtros rodam na ordem de registro, e o primeiro registrado é o mais externo. Filtros de grupo envolvem os do endpoint. Coloque os que podem curto-circuitar barato (autorização de recurso) antes dos que fazem trabalho (auditoria).

5.5 Serviços por feature

Cada feature registra suas próprias dependências, e só elas:

```

public static class ArquivosServicos
{
    public static IServiceCollection AddArquivos(this IServiceCollection
services)
    {
        services.AddScoped<IConsultaDeArquivos, ConsultaDeArquivos>();
        services.AddScoped<IAcessoAArquivos, ConsultaDeArquivos>();
        services.AddScoped<RecebimentoDeArquivos>();
        services.AddSingleton<IArmazenamento, ArmazenamentoS3>();
        return services;
    }
}

```

Remover uma feature passa a ser apagar uma pasta e duas linhas do `Program.cs`.

5.6 Quanta arquitetura em camadas?

Vertical slices não proíbem camadas; elas decidem onde cada camada vale a pena:

- **Leituras** (consultas, listagens) podem usar o `DbContext` direto, com projeção para o tipo de resposta. Um repositório genérico por cima do EF Core não acrescenta nada além de esconder `AsNoTracking`, `Select` e paginação, que são justamente o que importa para desempenho.

- **Escritas com regra de negócio** (receber arquivo, liquidar título) ganham um serviço ou um modelo de domínio, testável sem HTTP.
- **Domínio compartilhado** entre a API e os workers do primeiro curso (o `NossoNumero`, o `RegistroRetorno`, os status) mora num projeto separado, referenciado pelos dois.

Bibliotecas de mediador (despacho de comandos e consultas por um objeto central) são comuns em projetos .NET, mas não são necessárias com Minimal APIs: o handler já é o ponto de despacho. Se for adotar uma, verifique a licença; algumas das mais conhecidas passaram a exigir licença comercial.

5.7 Um detalhe de C# 14

O `ClienteId()` usado nos exemplos é um método de extensão comum. Com os *extension members* do C# 14, ele pode virar uma propriedade, o que lê melhor nos handlers:

```
public static class ClaimsPrincipalExtensions
{
    extension(ClaimsPrincipal usuario)
    {
        public Guid ClienteId => Guid.Parse(
            usuario.FindFirstValue("cliente_id")
            ?? throw new InvalidOperationException("Token sem a claim
cliente_id.));
    }
}

// uso: usuario.ClienteId (forma usada nos exemplos a partir do Módulo 6)
```

Exercício do Módulo 5

Reorganize o que você construiu até aqui na estrutura de pastas por feature, com `MapGrupoArquivos`, `MapGrupoTitulos`, `AddArquivos` e `AddTitulos`. Implemente `ExigirDonoDoArquivo` e aplique-o a `GET /{id}`. Teste que o arquivo de outro cliente devolve 404. Depois aplique o filtro a um endpoint sem parâmetro `id` e confirme que a aplicação falha na subida com uma mensagem clara.

Módulo 6 — Dados: consultas, paginação, streaming e upload

Numa API, o acesso a dados define a latência e a memória de cada requisição. As regras são as mesmas do curso de processamento de arquivos, aplicadas ao ciclo curto de uma requisição: projetar só o necessário, paginar por chave, transmitir em vez de materializar, receber uploads em streaming e tornar o `POST` idempotente.

6.1 Registrando o EF Core para uma API

```
builder.Services.AddDbContextPool<CobrancaDb>(o => o
    .UseNpgsql(builder.Configuration.GetConnectionString("Cobranca"), npgsql
=> npgsql
    .EnableRetryOnFailure(maxRetryCount: 3))
    .UseQueryTrackingBehavior(QueryTrackingBehavior.NoTracking));
```

Três escolhas:

- **AddDbContextPool** reaproveita instâncias de **DbContext** entre requisições, economizando a construção do objeto a cada chamada. A condição é não guardar estado próprio no contexto (campos com o cliente atual, por exemplo).
- **NoTracking como padrão**: numa API, a maioria das operações é leitura. Nas escritas, você pede rastreamento explicitamente com **AsTracking()** ou trabalha com **Add / Update**.
- **EnableRetryOnFailure** repete comandos que falham por erro transitório. Quando você abre transações manualmente, envolva o bloco inteiro em **db.Database.CreateExecutionStrategy().ExecuteAsync(...)**, para que o retry repita a transação completa e não só o último comando.

Para identificadores, prefira **UUID versão 7** (**Guid.CreateVersion7()**): eles começam com um timestamp, então inserções caem no fim do índice em vez de espalhadas, o que mantém o índice da chave primária compacto. E continuam não sequenciais o bastante para não permitir que um cliente adivinhe IDs alheios.

6.2 Consultas: projeção direto no tipo de resposta

```
public sealed class ConsultaDeArquivos(CobrancaDb db) : IConsultaDeArquivos,
IAcessoAArquivos
{
    public Task<ArquivoResposta?> ObterAsync(Guid clienteId, Guid id,
CancellationTokentoken ct) =>
        db.Arquivos
            .Where(a => a.ClienteId == clienteId && a.Id == id) // o
cliente está em toda consulta
            .Select(a => new ArquivoResposta(
                a.Id, a.NomeOriginal, a.Banco, a.Status, a.RecebidoEm,
                a.LinhasProcessadas, a.LinhasRejeitadas))
            .FirstOrDefaultAsync(ct);

    public Task<bool> PertenceAsync(Guid clienteId, Guid id,
CancellationTokentoken ct) =>
        db.Arquivos.AnyAsync(a => a.Id == id && a.ClienteId == clienteId,
ct); // usado pelo filtro do Módulo 5
}
```

O **Select** direto para o **record** de resposta faz o EF gerar um **SELECT** só com as colunas usadas, sem materializar a entidade. A entidade de banco nunca sai pela API: se ela aparecer no contrato, qualquer coluna nova vira campo público e qualquer renomeação quebra clientes.

6.3 Paginação keyset com cursor opaco

O Módulo 7 do primeiro curso mostrou por que **OFFSET** não escala. Numa API, keyset aparece como um **cursor opaco**: o cliente recebe uma string que não deve interpretar e a devolve para pedir a próxima página.

```
public sealed record PaginaDe<T>(IReadOnlyList<T> Itens, string?
ProximoCursor);

public static class Cursor
{
    public static string Codificar(long sequencia)
        => Base64Url.EncodeToString(BitConverter.GetBytes(sequencia));

    public static bool TryDecodificar(string? cursor, out long sequencia)
    {
        sequencia = long.MaxValue; // sem cursor: começa do
mais recente
        if (cursor is null) return true;
        Span<byte> bytes = stackalloc byte[8];
```

```

        if (!Base64Url.TryDecodeFromChars(cursor, bytes, out int n) || n !=
8) return false;
        sequencia = BitConverter.ToInt64(bytes);
        return true;
    }
}

public async Task<PaginaDe<ArquivoResumo>> ListarAsync(Guid clienteId,
FiltroDeArquivos f, long antesDe,
                                CancellationToken ct)
{
    var itens = await db.Arquivos
        .Where(a => a.ClienteId == clienteId && a.Sequencia < antesDe)
        .Where(a => f.Status == null || a.Status == f.Status)
        .Where(a => f.De == null || a.DataRecebimento >= f.De)
        .Where(a => f.Ate == null || a.DataRecebimento <= f.Ate)
        .OrderByDescending(a => a.Sequencia)
        .Take(f.Limite + 1) // um a mais
para saber se há próxima página
        .Select(a => new ArquivoResumo(a.Id, a.NomeOriginal, a.Status,
a.RecebidoEm, a.Sequencia))
        .ToListAsync(ct);

    bool temMais = itens.Count > f.Limite;
    if (temMais) itens.RemoveAt(itens.Count - 1);

    return new PaginaDe<ArquivoResumo>(itens, temMais ?
Cursor.Codificar(itens[^1].Sequencia) : null);
}

```

O handler de listagem, versão final do esboço do Módulo 3.5, decodifica o cursor antes de consultar e devolve 400 quando ele é inválido:

```

internal static async Task<Results<Ok<PaginaDe<ArquivoResumo>>,
ValidationProblem>> ListarArquivos(
    [AsParameters] FiltroDeArquivos filtro, ClaimsPrincipal usuario,
    IConsultaDeArquivos consulta, CancellationToken ct)
{
    if (!Cursor.TryDecodificar(filtro.Cursor, out long antesDe))
        return TypedResults.ValidationProblem(
            new Dictionary<string, string[]> { ["cursor"] = ["Cursor
inválido."] });

    return TypedResults.Ok(await consulta.ListarAsync(usuario.ClienteId,
filtro, antesDe, ct));
}

```


A coluna `sequencia`, a `data_recebimento` e o índice `(cliente_id, sequencia DESC)` vêm da tabela `arquivo` definida no Módulo 2.3 do curso de processamento de arquivos; a API e os workers compartilham o mesmo `CobrancaDb`. O campo `Sequencia` do `ArquivoResumo` existe só para montar o cursor: marque-o com `[JsonIgnore]` para que ele não vaze para o contrato.

Detalhes que fazem diferença:

- A coluna de paginação é uma `Sequencia bigint` com `IDENTITY`, crescente e única. Ela dá uma ordem total estável, que um timestamp sozinho não dá (dois arquivos no mesmo milissegundo).
- O índice certo é `(cliente_id, sequencia DESC)`: igualdade primeiro, ordenação depois.
- Pedir `Limite + 1` itens responde "existe próxima página?" sem um `COUNT(*)`, que em tabelas grandes é caro. Se o cliente precisar de total, ofereça um endpoint separado ou um valor aproximado.
- O cursor é opaco: hoje é uma sequência em Base64Url, amanhã pode carregar mais campos, sem mudar o contrato. Um cursor inválido gera 400.

6.4 Respostas em streaming

Um handler que devolve `IAsyncEnumerable<T>` é serializado como um array JSON escrito à medida que os itens chegam, sem montar a lista inteira em memória:

```
private static IEnumerable<OcorrenciaResposta> OcorrenciasDoTitulo(
    NossoNumero nossoNumero, ClaimsPrincipal usuario, CobrancaDb db) =>
    db.Ocorrencias
        .Where(o => o.ClienteId == usuario.ClienteId && o.NossoNumero ==
nossoNumero.Valor)
        .OrderBy(o => o.DataOcorrencia)
        .Select(o => new OcorrenciaResposta(o.Codigo, o.DataOcorrencia,
o.ValorPagoCentavos))
        .AsAsyncEnumerable();
```

Para exportações em CSV, escreva direto no corpo da resposta com `TypedResults.Stream`, que recebe um callback com o stream de saída:

```
private static IActionResult ExportarOcorrencias([FromQuery] DateOnly dia,
ClaimsPrincipal usuario, CobrancaDb db) =>
    TypedResults.Stream(async saida =>
    {
        await using var escritor = new StreamWriter(saida, new
UTF8Encoding(false), bufferSize: 64 * 1024);
        await
escritor.WriteLineAsync("nosso_numero;ocorrencia;data;valor_centavos");

        await foreach (var o in db.Ocorrencias
            .Where(x => x.ClienteId == usuario.ClienteId && x.DataOcorrencia
== dia)
            .Select(x => new { x.NossoNumero, x.Codigo, x.DataOcorrencia,
x.ValorPagoCentavos })
            .AsAsyncEnumerable())
        {
            await escritor.WriteLineAsync($"{o.NossoNumero};{o.Codigo};
{o.DataOcorrencia:yyyy-MM-dd};{o.ValorPagoCentavos}");
        }
    }, contentType: "text/csv", fileDownloadName: $"ocorrencias-{dia:yyyy-MM-
dd}.csv");
```

Três avisos sobre streaming. Depois que o primeiro byte sai, o status 200 já foi enviado: um erro no meio não vira mais um `ProblemDetails`, só uma conexão interrompida, então valide tudo antes de começar. A requisição pode ser longa, então dê a esse endpoint uma política de timeout própria (Módulo 9). E a leitura em streaming mantém uma conexão com o banco aberta durante toda a resposta; para exportações muito grandes, considere gerar o arquivo de forma assíncrona e devolver um link.

6.5 Upload de arquivos grandes

Existem três formas de receber um arquivo, e só uma serve para centenas de megabytes:

Forma	Como funciona	Quando usar
<code>IFormFile</code>	O framework lê o <code>multipart/form-data</code> inteiro e guarda em disco temporário antes de chamar o handler	Arquivos pequenos e formulários web
<code>MultipartReader</code>	Você lê as seções do multipart à mão, em streaming	Precisa de multipart (vários arquivos, campos) e streaming

Forma	Como funciona	Quando usar
Corpo cru (Stream)	O corpo inteiro é o arquivo (application/octet-stream); metadados vão em query ou headers	Integração máquina a máquina, arquivos grandes

Para a API de cobrança, o corpo cru é o mais simples e eficiente: o handler recebe o `Stream` da requisição e o repassa para o storage sem nunca ter o arquivo inteiro em memória ou em disco local.

```
grupo.MapPost("/", ReceberArquivo)
    .WithName("ReceberArquivo")
    .Accepts<Stream>("application/octet-stream")
    .WithMetadata(new RequestSizeLimitAttribute(500L * 1024 * 1024)) // só
este endpoint aceita 500 MB
    .RequireAuthorization("PodeEnviarArquivos");

private static async Task<Results<AcceptedAtRoute<ArquivoResposta>,
ProblemHttpResult>> ReceberArquivo(
    Stream corpo,
    [FromQuery, Required, StringLength(255)] string nomeArquivo,
    [FromQuery, Required, StringLength(10)] string banco,
    [FromHeader(Name = "Idempotency-Key"), StringLength(100)] string?
chaveIdempotencia,
    ClaimsPrincipal usuario,
    RecebimentoDeArquivos recebimento,
    CancellationToken ct)
{
    var resultado = await recebimento.ReceberAsync(
        new PedidoDeRecebimento(usuario.ClienteId, nomeArquivo, banco,
        chaveIdempotencia), corpo, ct);

    return resultado.Erro is { } erro
        ? erro.ParaProblema()
        : TypedResults.AcceptedAtRoute(resultado.Valor!, "ObterArquivo", new
{ id = resultado.Valor!.Id });
}
```

O limite de 500 MB vem do metadado `RequestSizeLimitAttribute` aplicado só a este endpoint; confirme com um teste que um corpo maior recebe 413. Se preferir deixar o controle explícito no código, ajuste

`IHttpMaxRequestBodySizeFeature.MaxRequestBodySize` no início do handler, antes de qualquer leitura do corpo. E lembre que o ingress tem o próprio limite, que precisa ser ajustado na mesma rota (Módulo 11).

Dentro de `RecebimentoDeArquivos.ReceberAsync`, o fluxo repete o serviço de ingestão do primeiro curso: ler o stream em blocos, calcular o SHA-256 incrementalmente e enviar as partes para o storage num upload multipart; só então, numa transação, inserir o registro do arquivo (com `UNIQUE` no hash) e a mensagem na outbox. Se o hash já existir, o objeto recém-enviado é descartado e o desfecho é `ArquivoDuplicado` (409).

Uma nota sobre antiforgery: desde o .NET 8, endpoints que fazem binding de formulário (`IFormFile`, `[FromForm]`) exigem token antiforgery por padrão, proteção pensada para navegadores com cookies. Numa API autenticada por token, chame `.DisableAntiforgery()` nesses endpoints. Com corpo cru isso não se aplica.

6.6 Idempotência no POST

A deduplicação por hash protege contra o mesmo arquivo enviado duas vezes, mas o cliente também precisa saber **qual foi a resposta** da tentativa que ele não viu (a conexão caiu antes). O header `Idempotency-Key` resolve isso:

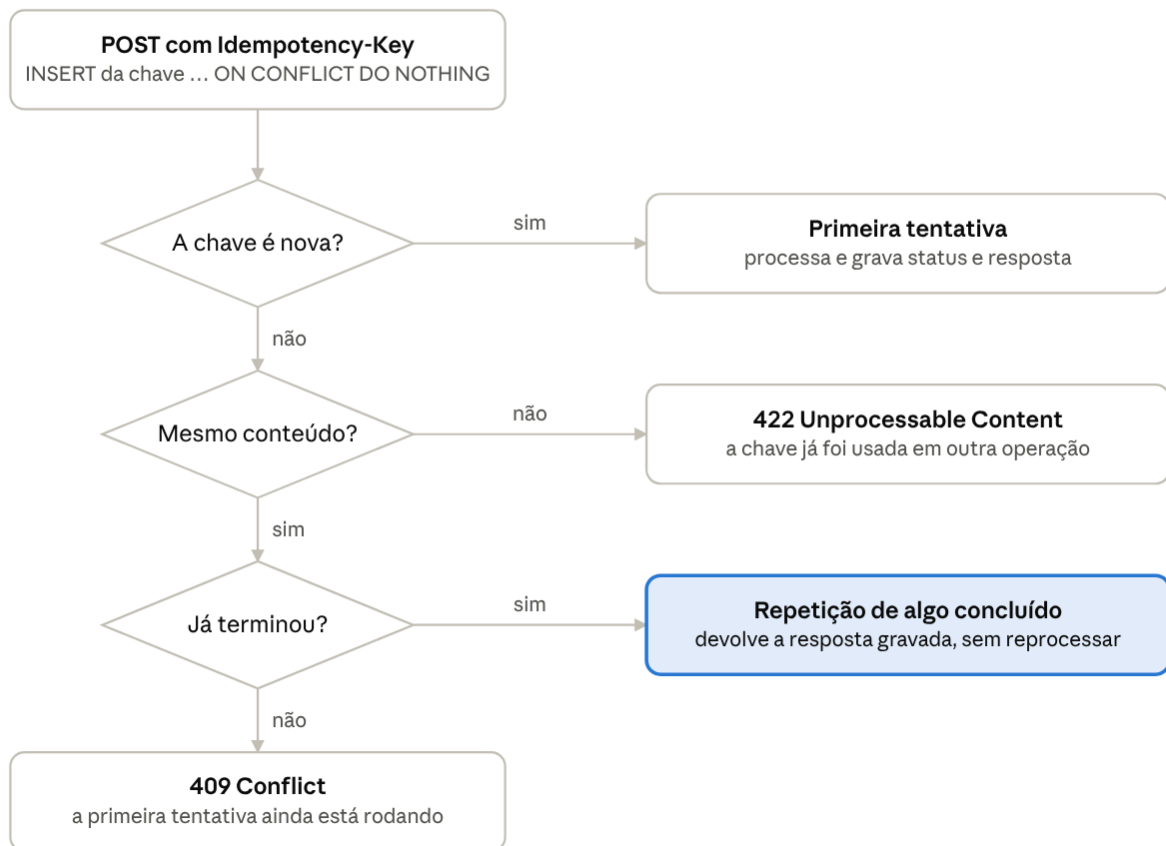
```
CREATE TABLE idempotencia (
    cliente_id      uuid      NOT NULL,
    chave           text      NOT NULL,
    hash_requisicao  text      NOT NULL,
    status          smallint   NULL,      -- NULL = em andamento
    resposta        jsonb      NULL,
    criado_em       timestamptz NOT NULL DEFAULT now(),
    PRIMARY KEY (cliente_id, chave)
);
```

O algoritmo:

1. `INSERT ... ON CONFLICT DO NOTHING` com a chave e o hash dos metadados da requisição (nome, banco).
2. Se inseriu, esta é a primeira tentativa: processe e, no fim, grave `status` e `resposta`.
3. Se não inseriu e a linha tem `status`, é uma repetição de algo que já terminou: devolva a resposta gravada, sem reprocessar.
4. Se não inseriu e o `status` é nulo, a primeira tentativa ainda está em andamento: devolva 409 e o cliente tenta de novo depois.
5. Se o `hash_requisicao` for diferente, o cliente reutilizou a chave para outra operação: 422.

Repetir a mesma chave devolve a mesma resposta, sem executar de novo

Tratamento do header Idempotency-Key (Módulo 6.6) · em destaque, a repetição



Idempotency-Key · 3 decisões, 4 desfechos

Só o primeiro caminho executa a operação; todos os outros respondem a partir do que já está gravado na tabela `idempotencia`.

Um job remove chaves com mais de 24 horas. Esse prazo é parte do contrato e deve estar documentado.

6.7 Concorrência otimista em atualizações

Quando dois clientes podem alterar o mesmo recurso, use concorrência otimista. No PostgreSQL, a coluna de sistema `xmin` muda a cada atualização da linha e serve como versão, sem coluna extra:

```

modelBuilder.Entity<Callback>()
    .Property(c => c.Versao)
    .IsRowVersion(); // o provedor Npgsql mapeia uma propriedade uint
IsRowVersion para xmin
  
```

Na API, exponha a versão como **ETag** no `GET` e exija `If-Match` no `PUT` / `PATCH`. Se a versão do `If-Match` não for a atual, o EF lança `DbUpdateConcurrencyException` e o tratador do Módulo 4 responde 409 (ou 412 Precondition Failed, que é o status específico para `If-Match` divergente).

Exercício do Módulo 6

Implemente `GET /v1/arquivos` com cursor `keyset` e confira com `EXPLAIN` que a página 100 custa o mesmo que a primeira. Implemente `POST /v1/arquivos` com corpo cru e envie um arquivo de 400 MB com `curl --data-binary`; acompanhe a memória do processo com `dotnet-counters` e confirme que ela não cresce com o tamanho do arquivo. Por fim, envie o mesmo upload duas vezes com a mesma `Idempotency-Key` e confirme que a segunda resposta é idêntica à primeira e que nada foi gravado de novo.

Módulo 7 — Contrato: OpenAPI e versionamento

O documento OpenAPI é o contrato da API em formato que máquinas entendem: gera clientes, alimenta portais, permite testes de contrato e mostra, num diff, se uma mudança quebra alguém. No ASP.NET Core 10 ele é gerado a partir dos metadados dos próprios endpoints, em OpenAPI 3.1, e o trabalho do desenvolvedor é garantir que esses metadados sejam completos e versionar com disciplina.

7.1 Ligando a geração

```
builder.Services.AddOpenApi();           // documento "v1" por padrão

app.MapOpenApi();                        // /openapi/v1.json
app.MapOpenApi("/openapi/{documentName}.yaml"); // também em YAML
```

O pacote `Microsoft.AspNetCore.OpenApi` percorre os endpoints, lê rotas, parâmetros, tipos de retorno e metadados, e monta o documento. No .NET 10 a saída padrão é **OpenAPI 3.1**, cujos schemas seguem JSON Schema 2020-12 (por exemplo, nulabilidade expressa como `"type": ["string", "null"]`, em vez do antigo `nullable: true`). A biblioteca interna passou para `Microsoft.OpenApi 2.0`, que mudou namespaces e alguns tipos; código de transformers escrito para o .NET 9 precisa de ajustes.

Decida conscientemente onde o documento fica exposto. Para APIs internas ou de parceiros, servir o documento é útil; para APIs públicas, alguns times preferem publicá-lo só num portal. Se for expor, coloque o `MapOpenApi()` sob autenticação ou restrinja por ambiente.

7.2 Metadados que fazem o documento ficar certo

Com `Results<...>` e `TypedResults` (Módulo 4), a maior parte das respostas já aparece sozinha. O que falta você completa no endpoint:

```
grupo.MapPost("/", ReceberArquivo)
    .WithName("ReceberArquivo") // operationId
    .WithSummary("Recebe um arquivo de retorno")
    .WithDescription("O arquivo é processado de forma assíncrona. Acompanhe pelo Location.")
    .Accepts<Stream>("application/octet-stream")
    .ProducesProblem(StatusCodes.Status409Conflict) // ProblemHttpResult
    não diz qual status
    .ProducesProblem(StatusCodes.Status422UnprocessableEntity)
    .ProducesProblem(StatusCodes.Status413PayloadTooLarge)
    .ProducesValidationProblem();
```

O `ProblemHttpResult` no tipo de retorno diz ao OpenAPI "pode haver um `ProblemDetails`", mas não com qual status; por isso cada status de erro relevante é declarado com `ProducesProblem`. Um bom critério: todo status que o cliente deve tratar de forma diferente aparece no documento.

7.3 Comentários XML viram documentação

No .NET 10, os comentários XML do código são lidos por um gerador de código e entram no documento OpenAPI. Ligue a geração do arquivo de documentação no `.csproj`:

```
<PropertyGroup>
  <GenerateDocumentationFile>true</GenerateDocumentationFile>
  <NoWarn>$(NoWarn);CS1591</NoWarn> <!-- não exige comentário em todo membro público -->
</PropertyGroup>
```

E documente onde a informação nasce:

```
/// <summary>Situação de um arquivo de retorno recebido.</summary>
/// <param name="Id">Identificador do arquivo, devolvido no recebimento.
</param>
/// <param name="LinhasRejeitadas">Linhas de detalhe que falharam na
validação do layout.</param>
public sealed record ArquivoResposta(
    Guid Id, string NomeOriginal, string Banco, StatusArquivo Status,
    DateTimeOffset RecebidoEm, int LinhasProcessadas, int LinhasRejeitadas);
```

A vantagem sobre `WithDescription` é que a documentação fica junto do tipo e é reaproveitada em todo endpoint que o usa.

7.4 Transformers: ajustando o documento

Transformers alteram o documento gerado. Existem três níveis: documento, operação e schema. O caso mais comum é declarar o esquema de autenticação:

```
using Microsoft.OpenApi;

builder.Services.AddOpenApi(o =>
{
    o.AddDocumentTransformer((documento, contexto, ct) =>
    {
        documento.Info = new OpenApiInfo
        {
            Title = "API de Cobrança",
            Version = "v1",
            Description = "Recebimento de arquivos de retorno e consulta de
títulos."
        };

        documento.Components ??= new OpenApiComponents();
        documento.Components.SecuritySchemes ??= new Dictionary<string,
IOpenApiSecurityScheme>();
        documento.Components.SecuritySchemes["Bearer"] = new
OpenApiSecurityScheme
        {
            Type = SecuritySchemeType.Http,
            Scheme = "bearer",
            BearerFormat = "JWT"
        };
        return Task.CompletedTask;
    });
});
```

Use transformers de operação para regras que valem para muitos endpoints (por exemplo, documentar o header `Idempotency-Key` em todo `POST` do grupo) e transformers de schema para ajustes de tipos (como dizer que o `NossoNumero` é uma string numérica com até 20 dígitos).

Declarar o esquema `Bearer` nos componentes não diz quais operações o exigem. Complete com um transformer de operação que adiciona o requisito de segurança nas operações cujo endpoint tem metadado de autorização (`IAuthorizeData`) e não tem `IAllowAnonymous` . Sem isso, ferramentas que geram clientes e a interface do Scalar não enviam o token.

7.5 Uma interface para explorar

Desde o .NET 9 os templates não incluem mais o Swagger UI. A opção mais usada hoje é o Scalar:

```
if (app.Environment.IsDevelopment())
{
    app.MapScalarApiReference();    // pacote Scalar.AspNetCore, em /scalar/v1
}
```

7.6 O documento no repositório

A prática que mais protege clientes: **gerar o documento no build e versioná-lo junto com o código**. O pacote `Microsoft.Extensions.ApiDescription.Server` gera o arquivo OpenAPI durante o `dotnet build`. Com o arquivo no repositório, todo pull request que muda o contrato mostra a mudança no diff, e uma ferramenta de comparação de OpenAPI no CI (como o `oasdiff`) pode falhar o build quando detecta mudança incompatível.

7.7 Versionamento

Estratégia	Exemplo	Prós	Contras
Caminho da URL	<code>/v1/arquivos</code>	Explícito, fácil de rotear, de cachear e de ler em log	URL muda entre versões
Header	<code>Api-Version: 2</code>	URL estável	Invisível em logs e no navegador; fácil de esquecer
Query string	<code>?api-version=2</code>	Simples de testar	Mistura versão com filtros
Media type	<code>Accept: application/vnd.cobranca.v2+json</code>	Mais "REST"	Complexo para clientes e ferramentas

Para APIs de integração, **caminho da URL** é a escolha mais pragmática, e combina naturalmente com route groups. Cada versão tem seu grupo e seu documento:

```

builder.Services.AddOpenApi("v1");
builder.Services.AddOpenApi("v2");

// Cada feature declara o caminho completo e a versão do próprio grupo
// (Módulo 5)
app.MapGrupoArquivos();           // MapGroup("/v1/arquivos").WithGroupName("v1")
app.MapGrupoArquivosV2();         // MapGroup("/v2/arquivos").WithGroupName("v2")

// Dentro de MapGrupoArquivosV2, só o que mudou ganha handler novo;
// o resto reaproveita os mesmos métodos estáticos da v1:
// grupoV2.MapGet("/{id:guid}", ObterArquivoV2); // novo contrato de
// resposta
// grupoV2.MapGet("/", ListarArquivos);           // igual à v1

```

O `WithGroupName` faz cada endpoint aparecer só no documento da sua versão (`/openapi/v1.json` , `/openapi/v2.json`). Se precisar de versão por header ou de regras mais elaboradas, a biblioteca `Asp.Versioning.Http` oferece isso para Minimal APIs.

7.8 O que quebra e o que não quebra

Crie uma versão nova só para mudanças incompatíveis. Todo o resto evolui dentro da versão atual.

- **Compatível:** novo endpoint; novo campo **opcional** na requisição; novo campo na resposta; novo parâmetro de query opcional; novo status de erro para uma situação que antes não existia.
- **Incompatível:** remover ou renomear campo; mudar tipo ou formato; tornar obrigatório algo que era opcional; apertar validação (o que era aceito passa a ser recusado); mudar o significado de um status code.
- **Zona cinzenta:** novo valor num enum de resposta. É compatível pelo contrato, mas quebra clientes que fazem `switch` exaustivo ou desserializam com enums estritos. Documente desde a v1 que enums podem ganhar valores, e oriente os clientes a tratarem valores desconhecidos.

Para aposentar uma versão, anuncie com antecedência e sinalize nas respostas da versão antiga com os headers `Deprecation` e `Sunset` (este com a data de desligamento). Monitore o tráfego por versão e por cliente (Módulo 11) para saber quem ainda falta migrar.

Exercício do Módulo 7

Ligue `AddOpenApi` , os comentários XML e o Scalar. Abra o documento e verifique, endpoint por endpoint, se todos os status que você devolve estão descritos. Configure a geração no build, comite o arquivo e faça uma mudança incompatível de propósito

(renomeie um campo de `ArquivoResposta`): o diff do pull request deve mostrar o problema. Depois crie `/v2/arquivos` com o campo renomeado e confira os dois documentos separados.

Módulo 8 — Segurança: quem pode chamar o quê, e quantas vezes

Segurança numa API se constrói em camadas: **autenticação** (quem é você?) por JWT emitido por um provedor de identidade, **autorização por política** (você pode usar esta operação?), **autorização por recurso** (este dado é seu?) e **limites** (quantas vezes?). O ponto de partida é negar tudo e liberar explicitamente.

8.1 Autenticação com JWT Bearer

A API não emite tokens; ela os valida. Os clientes (sistemas de parceiros, outros serviços) obtêm um token de acesso de um provedor de identidade (Entra ID, Keycloak, Cognito, Auth0) usando o fluxo **client credentials**, próprio para integração máquina a máquina, e o enviam no header `Authorization: Bearer <token>`.

```
builder.Services
    .AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(o =>
    {
        o.Authority = builder.Configuration["Auth:Authority"]; // ex.:
https://login.exemplo.com/realms/cobranca
        o.Audience = builder.Configuration["Auth:Audience"]; // ex.: api-
cobranca
        o.MapInboundClaims = false; // mantém os
nomes originais das claims (sub, scope)
        o.TokenValidationParameters.ClockSkew = TimeSpan.FromSeconds(30);
    });
```

O que acontece em cada requisição, passo a passo:

1. Na primeira vez, o middleware baixa os metadados do provedor (`.well-known/openid-configuration`) e as chaves públicas de assinatura (JWKS), e os mantém em cache.
2. Verifica a **assinatura** do token com essas chaves. Um token alterado ou assinado por outro emissor falha aqui.
3. Verifica o **emissor** (`iss` igual à `Authority`), a **audiência** (`aud` igual à `Audience`) e a **validade** (`exp` e `nbf`, com tolerância de relógio de 30 segundos).
4. Se tudo passa, monta o `ClaimsPrincipal` com as claims do token. Se algo falha, a requisição segue como anônima, e quem decide responder 401 é a autorização.

A validação da **audiência** é a mais esquecida e uma das mais importantes: sem ela, um token emitido para outra API da empresa, pelo mesmo provedor, seria aceito aqui.

`MapInboundClaims = false` evita que o handler renomeie claims para os URIs longos do padrão antigo da Microsoft (`sub` virando `http://schemas.xmlsoap.org/.../nameidentifier`), o que torna as políticas confusas.

O curso assume que o token traz uma claim `cliente_id` com o identificador do cliente no seu sistema. Ela não é padrão: configure-a no provedor (no Keycloak, um *mapper* de claim no client de cada parceiro; no Entra ID, um atributo customizado ou uma tabela de correspondência a partir do `azp / appid`). É ela que a extensão `ClienteId` do Módulo 5 lê, e é por ela que o rate limiting particiona.

No .NET 10, endpoints de API (os que retornam dados, não páginas) protegidos por autenticação por **cookie** passaram a responder 401 e 403 em vez de redirecionar para uma tela de login, o que antes era uma fonte frequente de respostas HTML inesperadas em clientes de API.

8.2 Seguro por padrão: a fallback policy

```
builder.Services.AddAuthorizationBuilder()
    .SetFallbackPolicy(new AuthorizationPolicyBuilder()
        .RequireAuthenticatedUser()
        .Build())
    .AddPolicy("PodeEnviarArquivos", p =>
        p.RequireEscopo("arquivos:escrever"))
    .AddPolicy("PodeConsultar", p => p.RequireEscopo("cobranca:ler"));
```

A **fallback policy** se aplica a todo endpoint que não declara nenhuma regra de autorização. Com ela, um endpoint novo esquecido sem `.RequireAuthorization()` exige autenticação mesmo assim. Endpoints públicos (health checks, documento OpenAPI) precisam dizer `.AllowAnonymous()` de forma explícita, onde qualquer revisor vê.

8.3 Escopos: uma armadilha comum

Muitos provedores emitem os escopos numa única claim `scope` com valores separados por espaço (`"cobranca:ler arquivos:escrever"`); outros emitem várias claims `scp` . Por isso `RequireClaim("scope", "arquivos:escrever")` falha quando o token tem mais de um escopo: ele compara o valor inteiro. Escreva o requisito uma vez, do jeito certo:

```
public static class EscopoExtensions
{
    public static AuthorizationPolicyBuilder RequireEscopo(this
    AuthorizationPolicyBuilder p, string escopo) =>
        p.RequireAssertion(ctx => ctx.User.Claims
            .Where(c => c.Type is "scope" or "scp")
            .SelectMany(c => c.Value.Split(' ',
    StringSplitOptions.RemoveEmptyEntries))
            .Contains(escopo, StringComparer.Ordinal));
}
```

8.4 Autorização por recurso: a falha número um

Na lista OWASP API Security Top 10, a primeira posição é **Broken Object Level**

Authorization: a API confere que o usuário está autenticado e tem o escopo certo, mas não confere se o objeto pedido é dele. Trocar `/v1/arquivos/{meu-id}` por `/v1/arquivos/{id-de-outro}` e receber o dado alheio é exatamente essa falha.

Há três defesas, e você deve usar pelo menos duas:

- **Filtro na rota**, como o `ExigirDonoDoArquivo` do Módulo 5: declarativo e visível.
- **Filtro em toda consulta:** toda query de dados de cliente inclui `ClienteId == usuario.ClienteId`, como nos exemplos do Módulo 6.
- **Filtro global no EF Core:** o EF Core 10 introduziu *named query filters*, que permitem vários filtros nomeados por entidade e desligar um específico quando necessário. Um filtro `PorCliente` aplicado a todas as entidades multi-cliente garante que nenhuma consulta esqueça a cláusula. Atenção: o filtro depende do cliente da requisição atual, o que conflita com `AddDbContextPool` (o contexto é reaproveitado entre requisições); nesse caso, use `AddDbContext` ou uma fábrica que configure o cliente a cada uso. No projeto deste curso ficamos com as duas primeiras defesas e mantemos o pool.

E lembre do Módulo 4: para objeto de outro cliente, responda **404**, não 403.

8.5 Rate limiting

O middleware de rate limiting do ASP.NET Core aplica limites por política e por partição. Numa API B2B, a partição natural é o cliente:

```
builder.Services.AddRateLimiter(o =>
{
    o.RejectionStatusCode = StatusCodes.Status429TooManyRequests;

    o.AddPolicy("porCliente", ctx =>
    RateLimitPartition.GetTokenBucketLimiter(
```

```

        partitionKey: ctx.User.FindFirstValue("cliente_id") ?? "anonimo",
        factory: _ => new TokenBucketRateLimiterOptions
        {
            TokenLimit = 100, // rajada máxima
            TokensPerPeriod = 100,
            ReplenishmentPeriod = TimeSpan.FromMinutes(1), // 100 por minuto
            QueueLimit = 0 // não enfileira:
        });

o.AddPolicy("uploadsSimultaneos", ctx =>
    RateLimitPartition.GetConcurrencyLimiter(
        partitionKey: ctx.User.FindFirstValue("cliente_id") ?? "anonimo",
        factory: _ => new ConcurrencyLimiterOptions { PermitLimit = 2,
            QueueLimit = 0 }));

o.OnRejected = async (contexto, ct) =>
{
    if (contexto.Lease.TryGetMetadata(MetadataName.RetryAfter, out
        TimeSpan espera))
        contexto.HttpContext.Response.Headers.RetryAfter =
            ((int)espera.TotalSeconds).ToString();

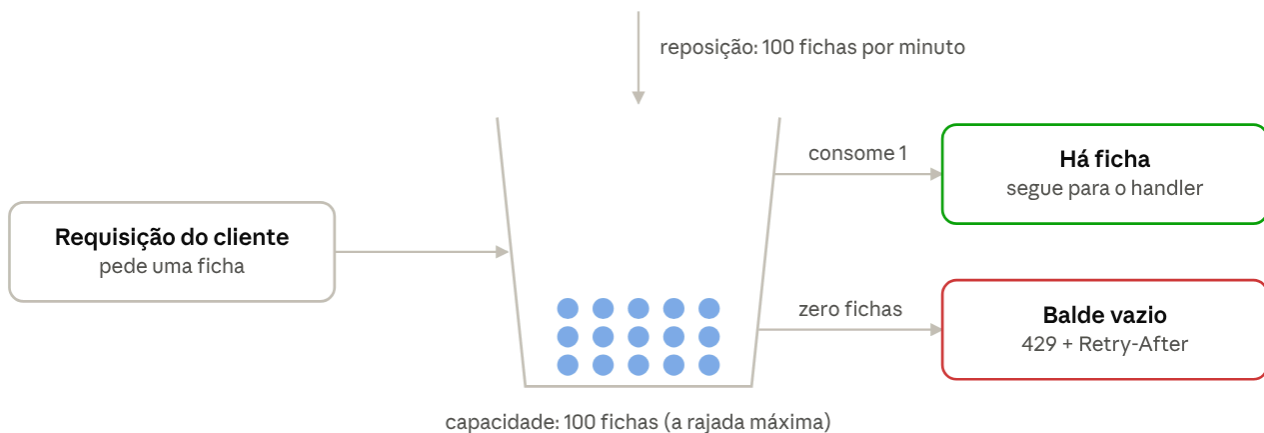
    await
        contexto.HttpContext.RequestServices.GetRequiredService<IProblemDetailsService>()
            .WriteAsync(new ProblemDetailsContext
            {
                HttpContext = contexto.HttpContext,
                ProblemDetails = { Title = "Limite de requisições excedido.",
                    Status = 429 }
            });
};
});

```

Por que cada algoritmo: o **token bucket** permite rajadas curtas (até 100 de uma vez) mas limita a taxa sustentada, que é o comportamento que integrações reais têm. O **limitador de concorrência** no upload limita quantos envios o mesmo cliente faz ao mesmo tempo, protegendo a banda e o storage independentemente de quantas requisições por minuto ele faça. O header `Retry-After` diz ao cliente quando tentar de novo, e o corpo segue o padrão `ProblemDetails`.

Cada requisição gasta uma ficha; sem ficha, a resposta é 429

Token bucket da política porCliente (Módulo 8.5) - um balde por cliente



token bucket · 100 fichas, reposição por minuto

Uma rajada de 100 chamadas passa de uma vez; a 101ª dentro do mesmo minuto encontra o balde vazio e recebe 429, com o tempo até a próxima reposição no `Retry-After`.

Para aplicar a política de concorrência, marque o endpoint de upload com `.RequireRateLimiting("uploadsSimultaneos")`. Cada endpoint usa uma única política, e a mais específica (a do endpoint) prevalece sobre a do grupo; o upload, então, troca o limite por minuto pelo de concorrência, que é o que importa para ele. Se precisar dos dois ao mesmo tempo, combine limitadores com `PartitionedRateLimiter.CreateChained` numa política própria.

Um limite importante: esses limitadores vivem **na memória de cada réplica**. Com 4 réplicas atrás de um balanceador, o limite efetivo é cerca de 4 vezes o configurado. Para limites contratuais exatos, aplique-os no API gateway ou use um limitador distribuído (Redis); o rate limiting da aplicação fica como proteção de cada instância.

8.6 CORS: só se houver navegador

CORS é uma regra que o **navegador** aplica a chamadas de JavaScript entre origens diferentes. Integrações servidor a servidor não são afetadas por ele. Se a API de cobrança também atende um portal web, libere só a origem do portal:

```
builder.Services.AddCors(o => o.AddPolicy("portal", p => p
    .WithOrigins("https://portal.cobranca.exemplo")
    .WithMethods("GET", "POST")
    .WithHeaders("Authorization", "Content-Type", "Idempotency-Key")
    .WithExposedHeaders("Location", "Retry-After")));
```

Nunca combine `AllowAnyOrigin` com credenciais, e lembre que CORS não é controle de acesso: ele não impede que um script fora do navegador chame a API.

8.7 O resto da lista

- **HTTPS** termina normalmente no ingress; garanta que nada chegue em HTTP puro de fora do cluster.
- **Limites de entrada:** tamanho de corpo por endpoint (Módulo 6), tamanho de query e de headers no Kestrel, timeouts (Módulo 9).
- **Segredos** (connection strings, chaves) vêm de um cofre ou de Secrets do Kubernetes injetados como variáveis de ambiente, nunca de `appsettings.json` comitado.
- **Logs sem dados sensíveis:** nunca registre tokens, e trate dados pessoais de pagadores (nome, documento) de acordo com a LGPD, mascarando-os em logs e traces.
- **Dependências:** rode `dotnet list package --vulnerable` no CI e atualize os patches mensais do .NET.

Exercício do Módulo 8

Suba um Keycloak em Docker, crie dois clientes de máquina com escopos diferentes e configure a API. Teste: sem token (401), token com audiência errada (401), token sem o escopo `arquivos:escrever` tentando enviar arquivo (403), cliente A lendo arquivo do cliente B (404), e 101 chamadas em um minuto (a última recebe 429 com `Retry-After`). Escreva cada caso como teste de integração no Módulo 10.

Módulo 9 — Performance sob carga

Quase toda a latência de uma API de negócio vem de I/O: banco, cache, serviços externos. As alavancas, em ordem de impacto, são: não repetir trabalho (cache), não esperar além do necessário (timeouts e cancelamento), proteger-se de dependências lentas (resiliência no `HttpClient`) e só então otimizar o processo em si (compressão, Native AOT). Meça antes de cada uma.

9.1 Medir primeiro

O número que importa é o **p95 ou p99 de latência sob carga realista**, não a média nem o tempo de uma chamada isolada. Uma API com média de 40 ms e p99 de 3 segundos tem 1% dos clientes esperando 3 segundos, e em integrações isso vira timeout e retry, que aumenta a carga. Use uma ferramenta de teste de carga (k6, NBomber, bombardier) com um perfil parecido com o real: proporção de leituras e escritas, distribuição de clientes, tamanhos de payload. O Módulo 11 mostra como obter esses percentis em produção.

9.2 Dois tipos de cache, para dois problemas

	Output caching	HybridCache
O que guarda	A resposta HTTP inteira	Qualquer objeto, sob uma chave sua
Onde atua	Middleware, antes do handler	Dentro do seu código
Bom para	Respostas iguais para todos (lista de bancos e layouts suportados)	Dados por cliente, resultados de consultas caras
Invalidação	Por tag ou por tempo	Por chave, por tag ou por tempo
Requisições autenticadas	Não cacheadas pela política padrão	Você controla a chave

O comportamento padrão do output caching de **não** cachear requisições autenticadas é uma proteção: uma resposta de um cliente servida a outro é um vazamento de dados. Para dados por cliente, o cache certo é o `HybridCache`, com o cliente na chave.

Output caching para dados públicos:

```
builder.Services.AddOutputCache(o =>
    o.AddPolicy("referencia", p =>
        p.Expire(TimeSpan.FromHours(1)).Tag("referencia")));

app.MapGet("/v1/layouts",
    ListarLayouts).AllowAnonymous().CacheOutput("referencia");

// quando um layout novo é publicado:
await outputCache.EvictByTagAsync("referencia", ct);
```

9.3 HybridCache

O `HybridCache` (pacote `Microsoft.Extensions.Caching.Hybrid`) combina um cache em memória local (L1) com um cache distribuído opcional (L2, como Redis) e resolve o problema mais traiçoeiro de cache: o **stampede**. Quando uma chave popular expira, cem requisições simultâneas não disparam cem consultas ao banco; uma consulta roda e as outras esperam o resultado dela.

```
builder.Services.AddHybridCache(o => o.DefaultEntryOptions = new
    HybridCacheEntryOptions
    {
        Expiration = TimeSpan.FromMinutes(5),           // L2
        LocalCacheExpiration = TimeSpan.FromMinutes(1) // L1, mais curto para
```

```

    limitar divergência entre réplicas
});
builder.Services.AddStackExchangeRedisCache(o => o.Configuration =
builder.Configuration["Redis"]); // vira o L2

public sealed class ConsultaDeTitulos(CobrancaDb db, HybridCache cache) :
IConsultaDeTitulos
{
    public ValueTask<TituloResposta?> ObterAsync(Guid clienteId, NossoNumero
numero, CancellationToken ct) =>
        cache.GetOrCreateAsync(
            $"titulo:{clienteId}:{numero}", // o cliente
            faz parte da chave
            (db, clienteId, numero),
            static async (estado, token) => await estado.db.Titulos
                .Where(t => t.ClienteId == estado.clienteId && t.NossoNumero
== estado.numero.Valor)
                .Select(t => new TituloResposta(t.NossoNumero, t.Situacao,
t.ValorCentavos, t.Vencimento))
                .FirstOrDefaultAsync(token),
            tags: [$"cliente:{clienteId}"],
            cancellationTokens: ct);
}

```

Repare na sobrecarga com **estado** ((db, clienteId, numero)) e lambda **static**: ela evita alocar uma closure a cada chamada. Quando o pipeline de arquivos atualiza um título, ele (ou o consumidor do evento da outbox) chama **cache.RemoveAsync(chave)** ou **RemoveByTagAsync** para invalidar.

Decida com o negócio quanto atraso é aceitável. Para "situação do título", 1 minuto de defasagem pode ser inaceitável logo após uma liquidação; nesse caso, invalide por evento em vez de confiar só na expiração.

Repare também que um título inexistente (resultado **null**) fica em cache como qualquer outro valor. Isso protege o banco de buscas repetidas por números que não existem, mas significa que, quando o título for criado, a chave precisa ser invalidada, ou o cliente verá 404 até a expiração.

9.4 Timeouts de requisição

Sem timeout, uma dependência lenta faz requisições se acumularem até esgotar conexões e memória. O middleware de request timeouts cancela o **CancellationToken** da requisição quando o prazo estoura (e, como todo handler o repassa, a consulta no banco também é cancelada):

```
builder.Services.AddRequestTimeouts(o =>
{
    o.DefaultPolicy = new RequestTimeoutPolicy { Timeout =
    TimeSpan.FromSeconds(10) };
    o.AddPolicy("exportacao", TimeSpan.FromMinutes(10));
});

app.UseRequestTimeouts();

app.MapGet("/v1/ocorrencias/export",
ExportarOcorrencias).WithRequestTimeout("exportacao");
app.MapGet("/v1/arquivos/{id:guid}/progresso",
Progresso).DisableRequestTimeout();
```

Por padrão, a requisição expirada recebe 504. Os timeouts não são aplicados com o depurador anexado, para não atrapalhar breakpoints, então teste-os com a aplicação rodando normalmente.

9.5 Chamando outros serviços

Toda chamada HTTP de saída passa por `IHttpClientFactory`, nunca por `new HttpClient()` a cada chamada (esgota portas) nem por um `HttpClient` estático eterno (ignora mudanças de DNS). O pacote `Microsoft.Extensions.Http.Resilience` adiciona um conjunto de estratégias prontas, ajustáveis por configuração:

```
builder.Services.AddHttpClient<ClienteDoAutbank>(c => c.BaseAddress = new
Uri(builder.Configuration["Autbank:Url"]!))
    .AddStandardResilienceHandler(); // limite de taxa, timeout total,
    retry com jitter, circuit breaker, timeout por tentativa
```

O **circuit breaker** é o que mais protege latência: quando a dependência está falhando, ele passa a recusar chamadas na hora por alguns segundos, em vez de fazer cada requisição esperar o timeout inteiro. A sua API responde rápido com erro (um 503 em `ProblemDetails`) em vez de devagar com erro.

9.6 Server-Sent Events: progresso em tempo real

O .NET 10 trouxe `TypedResults.ServerSentEvents`, que transmite um `IEnumerable<T>` como um stream de eventos SSE numa conexão HTTP comum. Para acompanhar o processamento de um arquivo, é mais simples que WebSockets e funciona através de proxies HTTP:

```

grupo.MapGet("/{id:guid}/progresso", Progresso)
    .ExigirDonoDoArquivo()
    .DisableRequestTimeout();

private static ServerSentEventsResult<ProgressoArquivo> Progresso(
    Guid id, IConsultaDeArquivos consulta, CancellationToken ct)
{
    return TypedResults.ServerSentEvents(Acompanhar(id, consulta, ct),
    eventType: "progresso");

    static async IAsyncEnumerable<ProgressoArquivo> Acompanhar(
        Guid id, IConsultaDeArquivos consulta, [EnumeratorCancellation]
        CancellationToken ct)
    {
        ProgressoArquivo? anterior = null;
        using var timer = new PeriodicTimer(TimeSpan.FromSeconds(1));

        do
        {
            ProgressoArquivo? atual = await consulta.ProgressoAsync(id, ct);
            if (atual is null) yield break;

            if (atual != anterior) // só envia quando algo mudou
            {
                yield return atual;
                anterior = atual;
            }

            if (atual.Status is StatusArquivo.Concluido or
            StatusArquivo.Rejeitado or StatusArquivo.Falhou)
                yield break; // estado final: encerra o
stream
        }
        while (await timer.WaitForNextTickAsync(ct));
    }
}

```

O `ProgressoArquivo` é um `record` (`public sealed record` `ProgressoArquivo(StatusArquivo Status, int LinhasProcessadas, int LinhasRejeitadas);`), e por isso `atual != anterior` compara valores, não referências. A consulta `ProgressoAsync` usa o `DbContext` da requisição, que continua válido enquanto o stream estiver aberto, porque o escopo só termina quando a resposta termina.

No navegador, o cliente é um `new EventSource(url)`; em C#, a classe `SseParser` de `System.Net.ServerSentEvents` lê o stream. Esta versão consulta o banco a cada segundo por conexão aberta, o que é aceitável para dezenas de conexões simultâneas. Para milhares, troque a consulta periódica por notificações (`LISTEN/NOTIFY` do PostgreSQL ou pub/sub do Redis, alimentado pelo worker). E configure o ingress para não fazer buffering da resposta e para ter um idle timeout maior que o intervalo entre eventos.

9.7 Compressão

Respostas JSON comprimem muito bem, e uma listagem de 200 KB pode cair para 20 KB. Ligue Brotli e Gzip:

```
builder.Services.AddResponseCompression(o =>
{
    o.EnableForHttps = true;
    o.Providers.Add<BrotliCompressionProvider>();
    o.Providers.Add<GzipCompressionProvider>();
});
```

Dois cuidados. Se o ingress já comprime, não comprima duas vezes; escolha um lugar. E `EnableForHttps` exige atenção ao ataque BREACH: não comprima respostas que misturem um segredo (como um token) com conteúdo controlado pelo cliente. Respostas de dados de negócio de uma API com bearer token normalmente não se encaixam nesse caso.

9.8 Native AOT: quando vale

Minimal APIs suportam publicação Native AOT: um binário nativo que sobe em milissegundos e usa bem menos memória. As exigências:

- `WebApplication.CreateSlimBuilder(args)` e `<PublishAot>true</PublishAot>`.
- JSON com source generation para todos os tipos de entrada e saída (Módulo 3).
- Nenhuma reflexão dinâmica: o Request Delegate Generator e o gerador de validação já cobrem os endpoints, mas bibliotecas de terceiros precisam ser compatíveis.
- EF Core ainda tem limitações em AOT; o caminho mais seguro é Npgsql direto ou Dapper com geração de código.

Para a API de cobrança, que usa EF Core e roda continuamente, JIT com ReadyToRun é a escolha pragmática: o Dynamic PGO do JIT tende a dar mais vazão em processos longos. AOT brilha em serviços pequenos que escalam de zero, em funções serverless e em sidecars.

9.9 As causas mais comuns de lentidão

- **N+1 consultas:** um laço que faz uma consulta por item. Projete tudo numa consulta só.
- **Ler o corpo inteiro para uma string** (`ReadToEndAsync`) em payloads grandes: aloca no LOH. Use binding direto ou streaming.
- **I/O síncrono ou `.Result`** : esgota o `ThreadPool` sob carga.
- **Esquecer o `CancellationToken`** : trabalho continua depois que o cliente desistiu.
- **Logar payloads inteiros** em nível `Information` em toda requisição.

Exercício do Módulo 9

Crie um teste de carga com k6 para `GET /v1/titulos/{nossoNumero}` com 50 usuários virtuais por 2 minutos e anote o p95. Adicione `HybridCache` e rode de novo. Depois simule um banco lento (`pg_sleep` numa consulta) e verifique que o timeout de 10 segundos devolve 504 e cancela a consulta no banco (confira em `pg_stat_activity`). Por fim, abra o endpoint de progresso no navegador com `EventSource` enquanto o worker do primeiro curso processa um arquivo.

Módulo 10 — Testes: provando que tudo funciona

O teste que mais vale numa API é o **teste de integração**: uma requisição HTTP de verdade, passando pelo pipeline inteiro (middlewares, autenticação, binding, validação, filtros), contra um PostgreSQL de verdade num container. O ASP.NET Core torna isso rápido com `WebApplicationFactory` , que sobe a aplicação em memória, e o `Testcontainers` cuida do banco.

10.1 O que testar em cada nível

Nível	O que cobre	Ferramenta	Quantidade
Unidade	Domínio, value objects (<code>NossoNumero</code>), handlers isolados	xUnit, sem servidor	Muitos, milissegundos cada
Integração	Pipeline HTTP + banco real: status, <code>ProblemDetails</code> , autorização, paginação	<code>WebApplicationFactory</code> + <code>Testcontainers</code>	Um por comportamento de endpoint
Contrato	O documento OpenAPI não mudou sem querer	Snapshot do <code>/openapi/v1.json</code>	Um por versão

Nível	O que cobre	Ferramenta	Quantidade
Carga	Latência e vazão sob carga	k6, NBomber	Poucos, fora do CI rápido

Repare que os testes de integração estão no centro, não no topo de uma pirâmide fina. Numa API, a maior parte dos bugs mora na ligação entre as peças (um middleware fora de ordem, um binding que não acontece, uma política que não é aplicada), e só um teste que atravessa o pipeline os pega.

10.2 Testes de unidade de handlers

Como os handlers são métodos estáticos que devolvem `TypedResults` (Módulos 4 e 5), eles são testáveis como funções comuns:

```
[Fact]
public async Task ObterArquivo_inexistente_devolve_404()
{
    var usuario = new ClaimsPrincipal(new ClaimsIdentity(
        [new Claim("cliente_id", Guid.NewGuid().ToString())],
        authenticationType: "Teste"));
    var consulta = new ConsultaFalsa(arquivo: null);

    var resultado = await ArquivosEndpoints.ObterArquivo(Guid.NewGuid(),
        usuario, consulta, CancellationToken.None);

    Assert.IsType<NotFound>(resultado.Result);
}
```

Para isso, deixe os handlers `internal` (em vez de `private`) e use `[assembly: InternalsVisibleTo("Cobranca.Api.Tests")]`.

10.3 A fábrica de testes de integração

No .NET 10, aplicações com top-level statements já expõem a classe `Program` para testes, sem a antiga declaração manual `public partial class Program;`. A fábrica abaixo sobe um PostgreSQL por classe de teste, aplica as migrations e troca a autenticação real por um esquema de teste:

```
public sealed class ApiFactory : WebApplicationFactory<Program>,
    IAsyncLifetime
{
    private readonly PostgreSQLContainer _postgres = new PostgreSQLBuilder()
        .WithImage("postgres:17-alpine")
        .Build();
```

```

public async ValueTask InitializeAsync()
{
    await _postgres.StartAsync();

    using var escopo = Services.CreateScope(); // força a
subida da aplicação
    await escopo.ServiceProvider.GetRequiredService<CobrancaDb>
().Database.MigrateAsync();
}

protected override void ConfigureWebHost(IWebHostBuilder builder)
{
    builder.UseEnvironment("Testing");
    builder.UseSetting("ConnectionStrings:Cobranca",
_postgres.GetConnectionString());

    builder.ConfigureTestServices(services =>
    {
        services.AddAuthentication(AutenticacaoDeTeste.Esquema)
            .AddScheme<AuthenticationSchemeOptions, AutenticacaoDeTeste>
(AutenticacaoDeTeste.Esquema, _ => { });

        services.RemoveAll<IArmazenamento>();
        services.AddSingleton<IArmazenamento, ArmazenamentoEmMemoria>();
    });
}

public HttpClient ClienteComo(Guid clienteId, string escopos)
{
    HttpClient http = CreateClient();
    http.DefaultRequestHeaders.Add(AutenticacaoDeTeste.HeaderCliente,
clienteId.ToString());
    http.DefaultRequestHeaders.Add(AutenticacaoDeTeste.HeaderEscopos,
escopos);
    return http;
}

public override async ValueTask DisposeAsync()
{
    await base.DisposeAsync();
    await _postgres.DisposeAsync();
}
}

```


O esquema de autenticação de teste lê o cliente e os escopos de headers e monta o mesmo `ClaimsPrincipal` que um JWT real produziria. As **políticas de autorização continuam as de produção**, que é o que importa testar:

```
public sealed class
AutenticacaoDeTeste(IOptionsMonitor<AuthenticationSchemeOptions> opcoes,
                    ILoggerFactory logger, UriEncoder
encoder)
    : AuthenticationHandler<AuthenticationSchemeOptions>(opcoes, logger,
encoder)
{
    public const string Esquema = "Teste";
    public const string HeaderCliente = "X-Teste-Cliente";
    public const string HeaderEscopos = "X-Teste-Escopos";

    protected override Task<AuthenticateResult> HandleAuthenticateAsync()
    {
        if (!Request.Headers.TryGetValue(HeaderCliente, out var cliente))
            return Task.FromResult(AuthenticateResult.NoResult()); // sem
header = anônimo

        Claim[] claims =
        [
            new("sub", cliente!),
            new("cliente_id", cliente!),
            new("scope", Request.Headers[HeaderEscopos].ToString())
        ];
        var identidade = new ClaimsIdentity(claims, Esquema);
        return Task.FromResult(AuthenticateResult.Success(
            new AuthenticationTicket(new ClaimsPrincipal(identidade),
Esquema)));
    }
}
```

Se quiser ir um passo além, uma alternativa é manter o `JwtBearer` real e configurá-lo nos testes com uma chave de assinatura gerada pelo próprio teste, emitindo tokens de verdade. Isso testa também emissor, audiência e expiração, ao custo de um pouco mais de código.

10.4 Testes que descrevem comportamento

Escreva um teste por comportamento do contrato, com o nome dizendo o comportamento:

```

public sealed class ArquivosTests(ApiFactory api) : IClassFixture<ApiFactory>
{
    private static readonly Guid ClienteA = Guid.CreateVersion7();
    private static readonly Guid ClienteB = Guid.CreateVersion7();

    [Fact]
    public async Task Cliente_nao_enxerga_arquivo_de_outro_cliente()
    {
        var ct = TestContext.Current.CancellationToken;
        HttpClient a = api.ClienteComo(ClienteA, "arquivos:escrever
cobranca:ler");
        HttpClient b = api.ClienteComo(ClienteB, "cobranca:ler");

        var conteudo = new ByteArrayContent(ArquivosDeExemplo.RetornoValido);
        conteudo.Headers.ContentType = new
MediaTypeHeaderValue("application/octet-stream");
        HttpResponseMessage envio = await a.PostAsync("/v1/arquivos?
nomeArquivo=ret.txt&banco=341", conteudo, ct);

        Assert.Equal(HttpStatusCode.Accepted, envio.StatusCode);
        Assert.NotNull(envio.Headers.Location);

        HttpResponseMessage consultaDoDono = await
a.GetAsync(envio.Headers.Location, ct);
        HttpResponseMessage consultaDeOutro = await
b.GetAsync(envio.Headers.Location, ct);

        Assert.Equal(HttpStatusCode.OK, consultaDoDono.StatusCode);
        Assert.Equal(HttpStatusCode.NotFound, consultaDeOutro.StatusCode);
    }

    [Fact]
    public async Task Arquivo_duplicado_devolve_409_com_problem_details()
    {
        var ct = TestContext.Current.CancellationToken;
        HttpClient a = api.ClienteComo(ClienteA, "arquivos:escrever");
        await EnviarAsync(a, ArquivosDeExemplo.RetornoValido2, ct);

        HttpResponseMessage segunda = await EnviarAsync(a,
ArquivosDeExemplo.RetornoValido2, ct);
        var problema = await
segunda.Content.ReadFromJsonAsync<ProblemDetails>(ct);

        Assert.Equal(HttpStatusCode.Conflict, segunda.StatusCode);
        Assert.Equal("application/problem+json",
segunda.Content.Headers.ContentType?.MediaType);
    }
}

```

```

        Assert.Equal("https://api.cobranca.exemplo/erros/arquivo-duplicado",
        problema!.Type);
        Assert.True(problema.Extensions.ContainsKey("traceId"));
    }
}

```

O `EnviarAsync` é um helper da própria classe, para não repetir a montagem do conteúdo em cada teste:

```

private static Task<HttpResponseMessage> EnviarAsync(HttpClient http, byte[]
arquivo, CancellationToken ct)
{
    var conteudo = new ByteArrayContent(arquivo);
    conteudo.Headers.ContentType = new
MediaTypeHeaderValue("application/octet-stream");
    return http.PostAsync("/v1/arquivos?nomeArquivo=ret.txt&banco=341",
    conteudo, ct);
}

```

Um cuidado: os testes usam as políticas reais de rate limiting, e uma suíte grande pode esbarrar no limite de 100 por minuto do mesmo cliente. Leia os limites da configuração e aumente-os no ambiente `Testing`, exceto no teste que verifica o 429, que usa um cliente próprio.

Os exemplos usam xUnit v3 (`TestContext.Current.CancellationToken`, `ValueTask` no `IAsyncLifetime`). O segundo teste verifica o contrato do erro inteiro: status, content type, type estável e `traceId`. É esse nível de detalhe que impede regressões silenciosas.

10.5 Isolamento entre testes

Testes que compartilham o banco precisam não interferir entre si. Três abordagens:

- **Dados únicos por teste:** cada teste usa seus próprios `ClienteId` e arquivos. Simples e permite rodar em paralelo; funciona bem quando toda consulta é filtrada por cliente, como nesta API.
- **Limpeza entre testes:** a biblioteca Respawn apaga os dados das tabelas rapidamente entre testes, preservando o schema.
- **Um container por classe:** a `IClassFixture` acima já faz isso. O custo é alguns segundos de subida por classe.

10.6 Teste de contrato com snapshot

Complemente o arquivo OpenAPI versionado do Módulo 7 com um teste que compara o documento gerado com um snapshot aprovado (a biblioteca `Verify` faz isso bem). Qualquer mudança no contrato faz o teste falhar e exige que alguém aprove o novo snapshot, de forma consciente, no pull request.

```
[Fact]
public async Task Contrato_v1_nao_mudou()
{
    string documento = await
        api.CreateClient().GetStringAsync("/openapi/v1.json",
        TestContext.Current.CancellationToken);
    await VerifyJson(documento);
}
```

10.7 Controlando o tempo

Regras que dependem do relógio (expiração de chaves de idempotência, janelas de rate limiting, prazos) ficam testáveis quando o código recebe um `TimeProvider` por injeção em vez de chamar `DateTimeOffset.UtcNow`. Nos testes, registre um `FakeTimeProvider` (pacote `Microsoft.Extensions.TimeProvider.Testing`) e avance o tempo com `Advance(TimeSpan.FromHours(25))` para verificar que a chave expirou, sem esperar 25 horas.

Exercício do Módulo 10

Monte o projeto de testes com a `ApiFactory` acima e transforme em testes todos os casos do exercício do Módulo 8 (401, 403, 404 de outro cliente, 429), mais: validação com 400 e o dicionário `errors`, paginação com cursor (a segunda página não repete itens da primeira) e idempotência (mesma chave, mesma resposta). Rode tudo no CI com Docker disponível e meça o tempo total; deve ficar abaixo de um minuto.

Módulo 11 — Operação: health checks, telemetria e deploy

Uma API em produção precisa responder a três perguntas a qualquer momento: está viva e pronta para receber tráfego (health checks), está atendendo bem (métricas por rota e status) e o que aconteceu com uma requisição específica (traces e logs correlacionados pelo `traceId`). O ASP.NET Core 10 já emite a maior parte desses dados; o trabalho é ligá-los, enriquecê-los com cuidado e fazer o deploy respeitar o ciclo de vida da aplicação.

11.1 Health checks: vivo x pronto

Para uma API, a diferença entre as duas sondas do Kubernetes é essencial:

- **Liveness** ("o processo está funcionando?"): se falhar, o pod é reiniciado. Deve checar só o próprio processo. Se ela depender do banco, uma queda do banco reinicia todas as réplicas em cadeia, o que piora a situação.
- **Readiness** ("posso receber requisições agora?"): se falhar, o pod sai do balanceamento, mas continua vivo. Aqui sim entram as dependências essenciais, como o banco.

```
builder.Services.AddHealthChecks()
    .AddNpgsql(builder.Configuration.GetConnectionString("Cobranca")!, tags:
["pronto"]);

app.MapHealthChecks("/health/live", new HealthCheckOptions { Predicate = _ =>
false }) // nenhum check: só "o processo responde"
    .AllowAnonymous();
app.MapHealthChecks("/health/ready", new HealthCheckOptions { Predicate = c
=> c.Tags.Contains("pronto") })
    .AllowAnonymous();
```

O `AddNpgsql` vem do pacote `AspNetCore.HealthChecks.Npgsql`. Os `AllowAnonymous()` são necessários por causa da fallback policy do Módulo 8. Mantenha os health checks fora do rate limiting e com resposta mínima, sem detalhes internos.

11.2 Métricas: o que já vem pronto

O ASP.NET Core emite métricas no padrão de convenções semânticas do OpenTelemetry. A mais importante é o histograma `http.server.request.duration`, com dimensões de método, **template de rota** (`/v1/arquivos/{id:guid}`, não a URL com o ID, o que mantém a cardinalidade baixa) e status code. Dele saem diretamente as três visões que um painel de API precisa: requisições por segundo, taxa de erro por status e p95/p99 por rota. O Kestrel emite métricas de conexões, e o .NET 10 acrescentou métricas de autenticação e autorização (quantas falham e por quê).

```
builder.Services.AddOpenTelemetry()
    .ConfigureResource(r => r.AddService("cobranca-api"))
    .WithMetrics(m => m
        .AddAspNetCoreInstrumentation()
        .AddHttpClientInstrumentation()
        .AddRuntimeInstrumentation()
        .AddNpgsqlInstrumentation())
    .WithTracing(t => t
        .AddAspNetCoreInstrumentation(o => o.Filter = ctx =>
!ctx.Request.Path.StartsWithSegments("/health"))
        .AddHttpClientInstrumentation()
        .AddNpgsql())
    .UseOtlpExporter();

builder.Logging.AddOpenTelemetry(o => { o.IncludeScopes = true;
o.IncludeFormattedMessage = true; });
```

O filtro nos traces descarta os health checks, que seriam milhares de spans inúteis por hora.

Para saber **qual cliente** consome mais ou sofre mais erros, dá para acrescentar uma dimensão à métrica de duração, por meio da feature `IHttpMetricsTagsFeature`:

```
app.Use(async (contexto, next) =>
{
    await next(contexto);
    if (contexto.User.FindFirstValue("cliente_id") is { } cliente)
        contexto.Features.Get<IHttpMetricsTagsFeature>
()?.Tags.Add(new("cobranca.cliente", cliente));
});
```

Registre esse middleware logo depois de `UseAuthentication`, para que o usuário já esteja identificado. Faça isso só se o número de clientes for pequeno (dezenas). Com milhares de clientes, a cardinalidade explode o custo do backend de métricas; nesse caso, a informação por cliente fica nos traces e nos logs.

11.3 Traces e logs correlacionados

Cada requisição vira um trace com spans filhos para cada consulta ao banco e cada chamada HTTP de saída. O `traceId` desse trace é o mesmo que vai no `ProblemDetails` (Módulo 4) e em todo log emitido durante a requisição. O fluxo de suporte fica direto: o cliente informa o `traceId` do erro, você abre o trace e vê qual consulta demorou ou qual dependência falhou.

O upload também conecta com o primeiro curso: se a mensagem da outbox carregar o contexto do trace (`traceparent`), o trace do `POST /v1/arquivos` continua no worker que processa o arquivo. Uma única linha do tempo mostra o arquivo da chegada até o último evento publicado.

Sobre logs de requisição: o middleware `HttpLogging` pode registrar cada requisição, mas com traces e métricas ligados ele raramente é necessário. Se usar, nunca registre corpos nem o header `Authorization`, e configure explicitamente quais campos entram.

11.4 Configuração que falha cedo

Configuração errada deve impedir a subida, não quebrar a primeira requisição:

```
public sealed class AuthOptions
{
    [Required, Url] public string Authority { get; init; } = "";
    [Required] public string Audience { get; init; } = "";
}

builder.Services.AddOptions<AuthOptions>()
    .BindConfiguration("Auth")
    .ValidateDataAnnotations()
    .ValidateOnStart();
```

Com `ValidateOnStart`, um `Auth:Audience` esquecido no ambiente de homologação derruba o pod no deploy, e o rollout para antes de atingir as réplicas saudáveis.

11.5 Desligamento gracioso

Quando o Kubernetes encerra um pod, duas coisas acontecem quase ao mesmo tempo: o pod é removido do balanceamento e recebe SIGTERM. Como a remoção do balanceamento leva alguns segundos para se propagar, requisições novas ainda podem chegar depois do SIGTERM. A solução clássica é um pequeno atraso antes de começar a desligar:

```
spec:
  terminationGracePeriodSeconds: 40
  containers:
    - name: api
      lifecycle:
        preStop:
          sleep: { seconds: 5 }      # deixa o balanceador parar de mandar
tráfego (ação sleep nativa: Kubernetes 1.30+)
```

Depois disso, o host do ASP.NET Core para de aceitar conexões e espera as requisições em andamento terminarem, até o `ShutdownTimeout` (configure-o abaixo do `terminationGracePeriodSeconds`). Conexões longas, como o stream de progresso, não terminam sozinhas: ligue o `IHostApplicationLifetime.ApplicationStopping` ao token do enumerador SSE para que ele encerre o stream e o cliente reconecte em outra réplica (o `EventSource` reconecta automaticamente).

11.6 Container e Kubernetes

Publique a imagem com o SDK (`dotnet publish /t:PublishContainer`). As imagens oficiais rodam como usuário não-root e escutam na porta 8080 por padrão (`ASPNETCORE_HTTP_PORTS`). Os pontos específicos de uma API, além do que o primeiro curso cobriu para workers:

- **Sondas:** `livenessProbe` em `/health/live` e `readinessProbe` em `/health/ready`. Para APIs, a readiness importa: é ela que tira uma réplica com problema do tráfego.
- **Autoscaling:** ao contrário do worker, uma API escala bem por CPU ou por requisições por segundo (HPA padrão ou KEDA com métrica do Prometheus). O teto continua sendo o pool de conexões do banco.
- **Ingress:** o proxy de entrada tem limites próprios, e eles ganham do Kestrel. Para o upload de 500 MB, aumente o limite de corpo do ingress (no NGINX Ingress, a anotação `proxy-body-size`) só na rota de upload; para exportações e SSE, aumente o timeout de leitura e desligue o buffering de resposta nessas rotas.
- **Forwarded headers:** com a API atrás do ingress, configure os proxies confiáveis no `ForwardedHeadersOptions` (Módulo 2), senão o rate limiting por IP e os logs verão o IP do ingress para todo mundo.

11.7 O painel e o primeiro diagnóstico

Um painel útil de API cabe numa tela: requisições por segundo; taxa de 5xx e de 4xx separadas (um pico de 401 é um cliente com credencial vencida, um pico de 500 é seu); p95 e p99 por rota; saturação (fila do ThreadPoo, uso do pool de conexões do Npgsql, CPU); e latência das dependências.

Quando o p99 sobe, siga esta ordem: a latência subiu em todas as rotas ou numa só? Se for numa, abra os traces lentos dessa rota e veja qual span domina. Se for em todas, olhe saturação: pool de conexões esgotado aponta para o banco ou para uma consulta nova sem índice; fila do ThreadPoo crescendo aponta para código bloqueante. Em poucos minutos, o problema sai de "a API está lenta" para uma consulta ou uma linha de código.

Exercício do Módulo 11

Ligue health checks, OpenTelemetry e o Aspire Dashboard em Docker. Rode o teste de carga do Módulo 9 e monte o painel da seção 11.7. Derrube o PostgreSQL e observe: a readiness falha, a liveness continua saudável e as respostas viram 503 rápidos em vez de timeouts. Por fim, provoque um erro 500 e encontre o trace dele a partir do `traceId` que voltou no ProblemDetails.

Projeto final — a API de cobrança, passo a passo

O projeto final é a API da tabela 1.4, construída em dez etapas, cada uma com um critério verificável por teste. Ela compartilha o domínio e o banco com o pipeline de arquivos do primeiro curso: a API recebe e consulta; os workers processam.

Estrutura da solução

```
Cobranca.sln
├─ src/
│   ├─ Cobranca.Dominio/           NossoNumero, StatusArquivo, ErroDeNegocio
│   (compartilhado com os workers)
│   ├─ Cobranca.Dados/           CobrancaDb, migrations, configurações do EF
│   Core
│   └─ Cobranca.Api/
│       ├─ Program.cs
│       ├─ Infraestrutura/       ProblemDetails, auth, rate limiting,
│   OpenAPI, telemetria
│       ├─ Comum/               paginação, cursor, extensões de
│   ClaimsPrincipal
│       └─ Features/
│           ├─ Arquivos/         upload, consulta, listagem, progresso SSE
│           ├─ Titulos/         consulta e ocorrências
│           └─ Ocorrencias/      exportação CSV
├─ tests/
│   ├─ Cobranca.Api.UnitTests/
│   ├─ Cobranca.Api.IntegrationTests/  ApiFactory + Testcontainers
│   └─ carga/                  scripts k6
└─ deploy/
    └─ docker-compose.yml       PostgreSQL, Redis, Keycloak, Aspire
Dashboard
    └─ k8s/                    Deployment, Service, Ingress, HPA, PDB
```

O Program.cs final

Depois das dez etapas, o `Program.cs` inteiro é este. Cada linha remete a um módulo:

```

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddProblemDetailsDaApi();           // Módulo 4: ProblemDetails
com traceId + IExceptionHandler
builder.Services.AddValidation();                   // Módulo 4
builder.Services.ConfigureJsonDaApi();              // Módulo 3: enums como
texto, source generation
builder.Services.AddOpenApiDaApi();                 // Módulo 7: v1, segurança
Bearer, comentários XML
builder.AddAutenticacaoEAutorizacao();              // Módulo 8: JWT, fallback
policy, escopos
builder.Services.AddRateLimitingDaApi();            // Módulo 8: por cliente e
uploads simultâneos
builder.Services.AddDesempenho();                   // Módulo 9: HybridCache,
output cache, timeouts, compressão
builder.AddDadosDaApi();                            // Módulo 6: EF Core com
pool, NpgsqlDataSource
builder.AddOperacao();                              // Módulo 11: health checks,
OpenTelemetry, options validadas

builder.Services.AddArquivos();                     // Módulo 5: uma linha por
feature
builder.Services.AddTitulos();
builder.Services.AddOcorrencias();

var app = builder.Build();

app.UseForwardedHeaders();                          // Módulo 2: a ordem do
pipeline
app.UseExceptionHandler();
app.UseStatusCodePages();
app.UseResponseCompression();                      // Módulo 9: remova se o
ingress já comprime
app.UseRequestTimeouts();
app.UseCors();
app.UseAuthentication();
app.UseTagDeClienteNasMetricas();                   // Módulo 11.2: opcional, só
com poucos clientes
app.UseAuthorization();
app.UseRateLimiter();
app.UseOutputCache();

app.MapOpenApi().AllowAnonymous();
app.MapHealthChecksDaApi();                         // /health/live e
/health/ready
app.MapGrupoArquivos();                             // cada grupo carrega rota

```

```

completa e versão (/v1/...)
app.MapGrupoTitulos();
app.MapGrupoOcorrencias();

app.Run();

```

Se alguém novo no time abrir só este arquivo, entende em um minuto o que a API tem e em que ordem as coisas acontecem. Esse é o teste de que a organização do Módulo 5 funcionou.

As dez etapas

- ☐ **1. Esqueleto.** Solução, `docker-compose`, `CobrancaDb` com migrations, `Program.cs` com `ProblemDetails` e health checks. Critério: `/health/ready` responde 200 com o banco no ar e 503 sem ele.
- ☐ **2. Primeiro endpoint de verdade.** `GET /v1/arquivos/{id}` com `Results<Ok<...>, NotFound>`, projeção direta e `ApiFactory` com `Testcontainers`. Critério: testes de integração para 200, 404 e GUID inválido.
- ☐ **3. Autenticação e autorização.** JWT com Keycloak, fallback policy, `RequireEscopo`, `ExigirDonoDoArquivo`. Critério: testes de 401, 403 e 404 para arquivo de outro cliente.
- ☐ **4. Listagem.** `GET /v1/arquivos` com `[AsParameters]`, validação embutida e cursor keyset. Critério: 400 com `errors` para filtros inválidos; páginas sem repetição; plano de consulta com índice.
- ☐ **5. Upload.** `POST /v1/arquivos` com corpo cru, limite de 500 MB só nesta rota, hash incremental, `UNIQUE` no hash, outbox. Critério: arquivo de 400 MB com memória estável; duplicado devolve 409 com `arquivoExistenteId`.
- ☐ **6. Idempotência.** Tabela `idempotencia` e o algoritmo do Módulo 6. Critério: mesma chave, mesma resposta; chave reutilizada com outro conteúdo, 422.
- ☐ **7. Títulos e exportação.** `GET /v1/titulos/{nossoNumero}` com `NossoNumero` e `HybridCache`; ocorrências em `IAsyncEnumerable`; exportação CSV com timeout próprio. Critério: exportação de 1 milhão de linhas com memória estável.
- ☐ **8. Progresso em tempo real.** SSE com `TypedResults.ServerSentEvents`, encerramento no estado final e no shutdown. Critério: o navegador acompanha um arquivo processado pelo worker do primeiro curso.
- ☐ **9. Contrato.** OpenAPI 3.1 com comentários XML, `ProducesProblem` em cada erro, documento gerado no build e teste de snapshot. Critério: renomear um campo quebra o teste de contrato.
- ☐ **10. Produção.** Rate limiting, OpenTelemetry, teste de carga com k6, deploy no Kubernetes com sondas, `preStop` e limites do ingress. Critério: p95 de consultas abaixo de 200 ms com 50 usuários virtuais, e um rollout durante o teste sem erros 5xx.

CrITÉRIOS de aceite finais

- ☐ Toda resposta de erro, de qualquer endpoint, é ProblemDetails com `type` estável e `traceId`
- ☐ Nenhum endpoint é acessível sem autenticação, exceto os marcados com `AllowAnonymous`
- ☐ Nenhum cliente consegue ler ou alterar dados de outro, e há teste para cada rota com ID
- ☐ O documento OpenAPI descreve todos os status que a API devolve e está versionado no repositório
- ☐ Upload, listagem e exportação usam memória constante, independentemente do volume
- ☐ Repetir um `POST` com a mesma `Idempotency-Key` nunca produz efeito duplicado
- ☐ Os testes de integração rodam contra PostgreSQL real em menos de um minuto
- ☐ O painel mostra RPS, erros e p95/p99 por rota, e todo erro é rastreável pelo `traceId`

Referências e próximos passos

Com os dois cursos concluídos, você tem as duas metades de um sistema de cobrança: a API que recebe e responde, e o pipeline que processa em volume. Os próximos passos naturais são design de APIs assíncronas e orientadas a eventos (AsyncAPI, CloudEvents), um API gateway na frente da API e, quando o .NET 11 for lançado em novembro, medir a mesma API nele antes de decidir migrar.

Documentação oficial por módulo

Módulo	Leitura
Visão geral	Novidades do ASP.NET Core 10 · Minimal APIs
3. Fundamentos	Binding de parâmetros
4. Respostas e erros	Respostas em Minimal APIs · Tratamento de erros em APIs · RFC 9457
5. Organização	Filtros em Minimal APIs
7. Contrato	OpenAPI no ASP.NET Core
8. Segurança	Rate limiting · OWASP API Security Top 10
9. Performance	HybridCache · Native AOT
10. Testes	Testes de integração · Testcontainers para .NET

Livros

- *API Design Patterns*, JJ Geewax: paginação, operações longas, idempotência e versionamento como padrões nomeados. Complementa os Módulos 1, 6 e 7.
- *Designing Web APIs*, Brenda Jin, Saurabh Sahni e Amir Shevat: o lado de produto de uma API (experiência do desenvolvedor, evolução, depreciação).

Fonte sobre as novidades do .NET 10

- [ASP.NET Core in .NET 10: Major Updates across Blazor, APIs, and OpenAPI](#) (InfoQ, 11/12/2025): validação embutida em Minimal APIs, tratamento de campos de formulário vazios, integração com `IProblemDetailsService`, Server-Sent Events, OpenAPI 3.1 com YAML e comentários XML, métricas de autenticação e o novo comportamento de cookies em endpoints de API.